SecretHandle: OS-Minted, Destination- and Task-Bound Opaque Secrets for Android Mobile AI Agents

Anonymous Author(s)[a]

[a]*Institution withheld for review, City, Country*

Abstract

Mobile graphical-user-interface agents must often use a one-time password, credential, address, phone number, or payment identifier to finish a task. Existing agents normally place the value in a screenshot, accessibility tree, prompt, or action argument, so an injected instruction can convert task authority into disclosure or redirection. We present SecretHandle, an Android design for *usable but invisible* secrets. A privileged service mints an opaque, authenticated handle at an authoritative OS source—the SMS or notification pipeline, Credential Manager, clipboard, or wallet—before the raw value reaches the agent. The handle carries a secret type, a task identifier, a destination predicate over package, origin, and field class, and a type-specific lifecycle. The agent observes the handle and issues `ENTER(handle)` toward a UI node; the OS resolves the value only after authenticating the handle, validating the task and target, enforcing expiry, use count, and confirmation policy, and reserving the handle against races. An observation redactor then masks the resolved field in accessibility and pixel channels.

We formalize mint, resolve, consume, expire, revoke, and redact as a transition system. Under an uncompromised reference monitor, authentic target identity, complete modeled-channel redaction, and a secure message authenticator, we prove conditional properties for agent-view non-disclosure, destination confinement, handle unforgeability, and single-use replay safety. The analysis deliberately excludes semantic misuse inside an allowed destination and side channels outside the modeled observation boundary. We implement an on-device Android prototype and evaluate it across OTP forwarding, destination redirection, replay, and race attacks against Claude Sonnet 4.6-driven Mobile-Agent v3.5 agents. SecretHandle achieves 0.00 observed exposure

(0/30, Wilson $[0.00, 0.11]$) versus 0.97 for unprotected agents (29/30), eliminates destination redirection leakage (0/30 versus 30/30 for unbound placeholders), and imposes sub-microsecond vault resolution latency (0.13 μs) with zero concurrency anomalies.

Keywords: mobile AI agents, Android security, opaque capabilities, destination binding, one-time passwords, prompt injection, observation redaction, trusted input

---

1. Introduction

Mobile agents increasingly operate applications by reading screenshots or accessibility trees and producing taps, swipes, and text-entry actions. Systems and benchmarks such as AppAgent, Mobile-Agent, AutoDroid, AndroidWorld, AndroidEnv, Android in the Wild, and SPA-Bench show why this interface is attractive: it reaches applications without requiring each developer to expose an agent-specific API [1, 2, 3, 4, 5, 6, 7]. The same interface collapses a security boundary. If a code or credential is visible enough for an agent to copy, it is visible enough for a compromised model, cloud endpoint, injected webpage, malicious notification, or logging layer to exfiltrate.

The risk is not hypothetical at the level of mechanism. Indirect prompt injection lets untrusted content compete with the user's instruction [8, 9, 10]; GUI-specific studies demonstrate that visually embedded or third-party content can steer agents toward privacy violations [11, 12, 13]. A recent mobile study exercises multi-app adversarial workflows and cross-app data exfiltration [14]. These results do not imply that every agent or every task fails, but they establish that a secret exposed to the planning loop is inside the attack surface.

Prompt filtering cannot provide the required invariant. A detector may miss a sensitive span; a placeholder may be substituted into the wrong field; and a model that never reads a value may still direct a local resolver to disclose it at an attacker-chosen destination. Mobile placeholder privacy is an important step: the closest design transforms sensitive screen content into type-preserving placeholders and mediates actions locally [15]. This motivates an abstraction boundary, but a security claim that begins after screen capture depends on detection coverage and does not itself bind the eventual value to the app, web origin, task, or field that is authorized to receive it.

This paper asks a narrower systems question: *can Android let an untrusted agent use a secret without ever giving the agent the secret or a transferable bearer copy?* Our answer is SecretHandle, an OS-minted opaque capability. Android already contains authoritative source points and relevant enforcement primitives: SMS Retriever routes app-associated verification messages [16]; Autofill and Credential Manager mediate user data for app fields [17, 18, 19]; Android 15 redacts detected OTP content from untrusted notification listeners and remote screen sharing [20]; and hardware-backed Keystore, key attestation, and Protected Confirmation support protected keys and high-assurance confirmation [21, 22, 23]. SecretHandle connects these ideas to the accessibility-agent action path.

At mint time, the trusted source pipeline gives the SecretHandle service a raw value, a type, provenance, and the narrowest defensible destination. The service stores the value in OS-private memory or sealed storage and returns a randomized identifier plus an authenticator. The agent receives only a typed description such as `OTP#7 expires in 43s; destination: com.example.bank/login_otp`. To act, it sends `ENTER(OTP#7)` and a target-node token. The resolver authenticates the handle, verifies the current task, package, WebView origin when trustworthy, field class, time, remaining uses, and confirmation rule, atomically reserves the record, and injects the value below the agent's observation boundary. The subsequent accessibility tree and screenshot contain a masked projection rather than the value.

Destination binding resembles audience restriction, not encryption for the model. OAuth resource indicators prevent a token intended for one resource from being replayed at another [24]; DPoP additionally constrains a token to a presenter key [25]. WebAuthn scopes credentials to relying parties [26]. SecretHandle applies a related principle to local UI data flow: possession of a handle is insufficient unless the OS-observed target and task match the handle policy. This analogy guides the design but does not make an OAuth token, passkey, or payment token an empirical baseline for Android text entry.

1.1. Contributions

This manuscript makes five scoped contributions.

1. It defines an OS-minted secret-handle abstraction for Android in which the raw value is intercepted at an authoritative source and represented to a mobile AI agent by a non-semantic, authenticated capability.

2. It designs a destination- and task-bound resolver for `ENTER(handle)` with atomic use reservation, typed lifecycles, optional protected confirmation, and explicit failure semantics. The destination includes package identity and may include a trustworthy web origin and field class; this granularity is a policy decision, not assumed ground truth.

3. It integrates observation-side redaction across accessibility and screenshot channels and identifies the closure conditions required to keep a just-entered value out of the next planning observation.

4. It supplies a formal transition system and conditional proofs for non-disclosure in the modeled agent view, destination confinement, reduction-style unforgeability, and single-use/expiry safety. The results do not claim to stop malicious actions that remain within an authorized destination.

5. It gives an implementation blueprint and preregistration-style evaluation plan for AOSP devices, login/2FA/checkout applications, real mobile agents, redirection and replay attacks, ablations, leakage instrumentation, and latency measurement. No unmeasured number is presented as a result.

1.2. Scope and paper organization

The trusted computing base includes the Android kernel and relevant framework services, the source adapters, the SecretHandle store and resolver, the redaction paths, and protected key operations. The agent, model provider, third-party content, and ordinary applications are untrusted. A compromised OS, a covert camera, timing leakage, or a malicious authorized destination is outside the central guarantee and is discussed in Section 11. The rest of the paper separates prior work (Section 2), notation and platform context (Section 3), the model and properties (Sections 4 and 5), the design and algorithms (Sections 6 and 8), formal analysis (Section 7), and the planned evaluation (Section 9).

2. Related work and novelty boundary

2.1. Mobile GUI agents and benchmarks

AppAgent uses multimodal interaction and experience learned through exploration or demonstrations [1]. Mobile-Agent emphasizes vision-centric perception without relying on app XML, while Mobile-Agent-v2 separates

planning, decision, and reflection and Mobile-Agent-v3 broadens the framework across GUI environments [2, 27, 28]. AutoDroid combines dynamic app analysis, UI representation, memory, and query optimization for Android task automation [3]. ScreenAgent and OSWorld expand the design and evaluation space for visual computer control [29, 30].

Android in the Wild provides large-scale demonstrations; AndroidEnv provides an interactive reinforcement-learning platform; AndroidWorld supplies reproducible state setup and programmatic task checks; and SPA-Bench integrates multiple smartphone agents and metrics [6, 5, 4, 7]. These works establish observation and action interfaces that SecretHandle must support. They do not claim an OS primitive for using an OTP or credential while withholding it from the agent. Moreover, AndroidWorld's standard applications avoid login-dependent tasks, which is why the proposed evaluation adds purpose-built, open authentication and checkout flows rather than claiming existing benchmark coverage.

2.2. Placeholders and inference-time privacy

The closest prior direction is privacy-preserving transformation of agent-visible content. Anonymization-Enhanced Privacy Protection for Mobile GUI Agents detects sensitive content, replaces it with type-preserving placeholders, transforms multiple UI representations, and resolves certain local interactions through a proxy [15]. This demonstrates that semantic placeholders can preserve task utility. SecretHandle changes the trust boundary in two ways: the handle is minted at a source that already owns or receives the value, so protection of integrated sources does not depend on finding the value after rendering; and use is authorized against an OS-observed destination rather than merely replacing a placeholder at a requested index.

Text and prompt privacy systems address a neighboring boundary. Casper detects and removes sensitive prompt content locally [31]; PAPILLON coordinates local and remote models for privacy-conscious delegation [32]; PrivacyRestore removes privacy spans and transmits a restoration representation in a client-server inference design [33]; and resource-aware work estimates when sanitization will harm utility [34]. These techniques can reduce cloud disclosure, but a mobile agent still needs a trusted actuation path when the hidden value must be entered into an application. We therefore treat sanitization as complementary and do not cite it as evidence for destination confinement.

2.3. Prompt injection, agent isolation, and information flow

Indirect prompt injection is an architectural risk when an agent interprets untrusted data as instructions [8]. InjecAgent and AgentDojo provide structured tests of attacks and defenses for tool-integrated agents [9, 10]. Fine-print and environmental injection studies extend this concern to visual and GUI environments [11, 12, 13]; mobile-agent experiments show that third-party channels can participate in realistic adversarial workflows [14].

Architectural defenses seek guarantees outside the language model. CaMeL extracts control and data flow from a trusted query and uses capabilities at tool calls [35]. Fides assigns confidentiality and integrity labels in a formal planner model [36]. Progent enforces privilege policies for agents [37]. IsolateGPT separates apps and their interactions, while AirGapAgent limits task context to reduce disclosure under context hijacking [38, 39]. SecretHandle shares the premise that a vulnerable model should not be the final policy authority. Its distinct object is a value that must cross from an OS source into a concrete UI field without crossing the planner's observation boundary.

2.4. Capabilities, mobile information flow, and Android mediation

Capabilities are unforgeable tokens of authority rather than descriptive names alone [40]. Least privilege, complete mediation, fail-safe defaults, and economy of mechanism remain useful design criteria [41]. Capsicum demonstrates a practical capability model for compartmentalizing commodity UNIX applications [42]. SecretHandle is not a general-purpose object-capability OS; it uses a narrow capability-shaped record for one actuation right, including explicit attenuation by destination, task, time, and use count.

Android security research has shown both platform mediation and its limits. Android Permissions Demystified measured the mapping between API behavior and permissions [43]. Fine-grained mandatory access-control work and SEApp illustrate the value and engineering cost of adding policies below applications [44, 45]. TaintDroid tracks sensitive flows dynamically from sources to sinks, while DroidScope reconstructs OS and runtime semantics for analysis [46, 47]. SecretHandle differs from taint tracking: it aims to prevent the raw value from entering an untrusted agent-visible representation, then authorizes one explicit sink. Tainting remains useful as an evaluation oracle for detecting accidental flows.

2.5. Authentication and bound tokens

HOTP and TOTP standardize common one-time-password constructions [48, 49]; NIST guidance describes authenticator and lifecycle considerations [50]. These standards define or govern codes, not how an accessibility agent can enter a code without seeing it. Android's SMS Retriever can route a verification message associated with an app, and OTP Autofill can mediate user-approved filling [16, 19]. Credential Manager and Autofill are primarily app- and user-driven platform interfaces [18, 17]. SecretHandle reuses their source knowledge where available but adds an agent action that never returns the value to the agent.

Audience restriction in OAuth prevents a token intended for one protected resource from being replayed at another, and DPoP adds proof-of-possession sender constraint [24, 25, 51]. WebAuthn scopes public-key credentials to relying parties [26]. These systems motivate narrow binding and explicit validation. Their network principals and protocols differ from a package/origin/field target inside Android; consequently the paper claims a design analogy, not protocol equivalence.

2.6. Novelty statement

Within the verified literature above, we did not find a system that simultaneously (i) mints an agent-facing opaque handle at an Android authoritative source before screen exposure; (ii) binds resolution to the current task and OS-observed UI destination; (iii) enforces a type-specific lifecycle; and (iv) closes both accessibility and pixel observation after entry. This is a bounded literature statement, not a claim of exhaustive priority. The closest mobile-placeholder work is newer than the original concept and materially narrows the novelty boundary; any submission must re-run the search and compare implementations and threat models in detail.

3. Platform background and notation

3.1. Android observation and actuation

An accessibility-based agent typically obtains a screenshot and a structured tree containing nodes, bounds, roles, text, content descriptions, and action affordances. Android accessibility services are explicitly privileged components that can observe events and act on behalf of users [52]. A screenshot-only agent may omit the tree, while a hybrid agent aligns visual

regions with nodes. In either case, ordinary text entry exposes a string to at least the action-producing process and frequently to logs or prompts.

Android Autofill has a different interaction pattern. The framework connects a client view, the system, and an Autofill service; the client exposes structure and accepts a selected dataset [17, 19]. Credential Manager aggregates passkeys, passwords, and federated sign-in credentials through app requests and user-facing selectors [18]. These mechanisms are useful precedents because the OS mediates sensitive data, but an autonomous accessibility agent generally cannot assume that the target app has initiated the appropriate request or that every relevant secret type has a platform provider.

Android already distinguishes some sensitive sources. The SMS Retriever flow delivers an app-associated verification message to a registered app and expects server-side verification [16]. User Consent allows a one-time, user-mediated SMS read [53]. The clipboard sensitivity extra is a rendering hint rather than an access-control guarantee [54]. Android 15 adds OTP redaction for many untrusted notification listeners and screen-sharing protections [20]. SecretHandle treats these as source-specific signals and enforcement hooks, not as proof that the unified design already exists.

3.2. Cryptographic and capability terminology

Let λ denote the security parameter and $F_K : \{0,1\}^* \to \{0,1\}^\lambda$ a message authentication function keyed by K. The implementation may use an approved HMAC construction [55] with a non-exportable key in Android Keystore [21]; the model needs only existential unforgeability under chosen-message attack for the authenticated handle representation. A capability conveys authority. A printed string that merely looks like a handle is not a capability unless the resolver authenticates it and finds a live server-side record.

We distinguish four identities: package identity p, web origin o, field class f, and task identity τ. A destination policy d is a predicate over a target context rather than necessarily a single tuple. For a native app, d may restrict p and f. For a WebView, it may also restrict o, but only if the platform can authenticate the displayed document origin. If reliable origin extraction is unavailable, policy must fail closed or require confirmation; it must not silently downgrade to a package-only guarantee.

Table 1: Principal notation. Every symbol used in the core transition system is defined here.

Symbol	Meaning
$s \in \mathcal{S}$	Raw secret value, never intentionally included in the agent view.
$q \in \mathcal{Q}$	Secret type: OTP, credential, identifier, clipboard value, or payment token.
$\sigma \in \mathcal{P}$	Authoritative source and provenance statement.
$\tau \in \mathcal{T}$	Unforgeable or OS-authenticated task identifier.
$d \in \mathcal{D}$	Destination predicate over package, origin, field class, and optional app-defined context.
$n \in \{0,1\}^\lambda$	Fresh random handle identifier.
t_i, t_e	Issue and expiry times, with $t_i < t_e$.
$u \in \mathbb{N}$	Remaining successful-use budget.
$c \in \mathcal{C}$	Confirmation policy, e.g., none, biometric, or Protected Confirmation.
$H = (n, q, \sigma, d, \tau, t_i, t_e, u, c)$	Authenticated handle metadata.
$a = F_K(\mathsf{enc}(H))$	Handle authenticator.
$h = (n, q, a)$	Compact agent-visible handle; non-sensitive display metadata may accompany it.
$x = (p, o, f, v)$	Resolver-authenticated target context for node token v.
Ω	Trusted SecretHandle store mapping n to secret and state.
$\mathsf{View}_A(\pi)$	Agent-visible projection of execution trace π.
ℓ	Explicit leakage allowed by the interface: type, coarse destination label, age/expiry, and status.

3.3. Lifecycle classes

Lifecycle is part of the authority, not advisory metadata. OTP handles default to one successful use and a short TTL matched to the issuer's validity window. Identifier handles may permit multiple uses only inside the same task and destination set. Credential handles are scoped to a relying app or origin and normally require a platform credential-selection event. Payment-

class handles additionally require per-use confirmation. A policy profile may always be tightened after minting; widening a destination or use budget requires a newly authenticated handle and, for sensitive classes, fresh user approval.

Table 2: Default policy profiles. These are design defaults to validate experimentally, not platform measurements.

Type	Source	Binding	Use budget	Confirmation
OTP	SMS/ notification	package + OTP field + task	1; short TTL	none/ user-consent
Credential	Credential Manager	package/origin + task	1 per request	platform selector
Identifier	clipboard/ profile	package + field class + task	bounded task use	policy dependent
Payment	wallet/token provider	package/origin + checkout task	1	per-use protected UI

3.4. Design principles

The design follows four rules. First, *mint before observation*: a protected source never emits the raw value onto an agent-readable path. Second, *authorize the sink*: the target is checked from trusted framework state, not from the model's textual assertion. Third, *consume atomically*: concurrent calls cannot both pass a single-use check. Fourth, *close the observation loop*: resolving a value is not sufficient if the next tree or screenshot reveals it.

These principles refine least privilege and complete mediation [41]. They also expose a limitation: a source not integrated with the OS cannot receive detector-independent protection. Such content may still use a detector-based placeholder, but its guarantee must be stated separately.

4. System and threat model

4.1. Entities and trust boundary

The trusted computing base (TCB) contains the Android kernel, the relevant portion of `system_server`, the SecretHandle service, source adapters, the resolver and input-dispatch integration, accessibility and screenshot redactors, and the keystore operation used to authenticate records. A

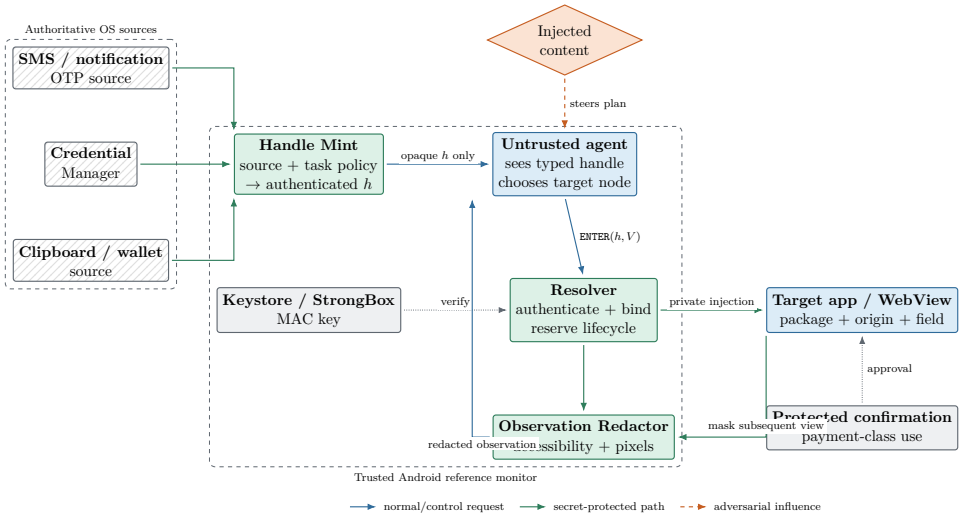


Figure 1: SecretHandle architecture. The raw value travels only from an authoritative source through the trusted mint/resolver path to an accepted destination. The agent receives an opaque handle and a redacted post-entry view. Colors and line styles distinguish ordinary, protected, and adversarial flows.

payment confirmation path may place the confirmation UI and key authorization inside a TEE; Android Protected Confirmation is optional at the device level and therefore must be detected rather than assumed [23].

The agent process, local or remote model, model provider, agent memory, ordinary applications, rendered web content, notifications not authenticated by a protected source, and app-supplied labels are untrusted. A target application may be honest, malicious, or confused. SecretHandle's destination property says whether a handle may be resolved at a target identity; it does not prove that an allowed app will subsequently use the value honestly.

The user authorizes a task and may approve sensitive mint or resolve events. The task broker gives the service a task identifier τ and an initial destination policy. The mechanism assumes τ cannot be chosen or modified by the agent alone. This can be implemented by an OS-signed task token or by state held in the service. The concept document mentions composition with a separate task-capability mechanism; because that artifact is not present, the manuscript specifies the minimum task-token contract independently rather than relying on an unavailable result.

4.2. Authoritative sources

An authoritative source is a TCB component or mediated provider that receives the secret before an agent-readable representation and can supply trustworthy provenance. The initial adapters are:

- **SMS/notification OTP adapter.** It observes a verification message in the protected delivery or classification path, derives an app binding when the message or request supports one, and withholds the code from the agent-facing notification. App-hash association is evidence for an app, not evidence for an arbitrary field.
- **Credential Manager adapter.** It mints only after the user selects a credential for a requesting app or origin. The adapter retains the credential material inside the provider/system path and exports a one-use handle.
- **Clipboard adapter.** It mints only for clips explicitly marked or user-designated as sensitive. Because `EXTRA_IS_SENSITIVE` does not itself change clipboard access control [54], SecretHandle must suppress or replace agent-visible clip content in addition to reading the hint.
- **Wallet adapter.** It accepts a token or payment credential from a wallet/provider contract and requires confirmation. Server-side payment tokenization is complementary; the local handle controls which UI destination receives the wallet output.

An OCR or named-entity detector is not an authoritative source. It may create a best-effort legacy handle with a visibly weaker assurance label, but such a path is excluded from the detector-independent theorems.

4.3. Adversary capabilities

The adversary controls the agent's planning output and may arrange arbitrary untrusted text or pixels in an app, webpage, notification, or message. It can request actions on any node visible to the agent, replay previously observed handles, guess new handle strings, race multiple `ENTER` calls, switch foreground apps, navigate a WebView, and render a string with the same visual syntax as a handle. It may read all agent inputs, outputs, prompts, tool traces, and application-visible values after legitimate injection if the destination app itself is compromised.

The adversary cannot read TCB memory, forge Android package identity, extract the non-exportable handle-authentication key, modify the resolver's atomic state transition, compromise the trusted display used for Protected Confirmation, or bypass every screenshot/accessibility redaction hook. These are explicit assumptions. A result that depends on them is a conditional system theorem, not an unconditional statement about all Android devices.

4.4. Security goals

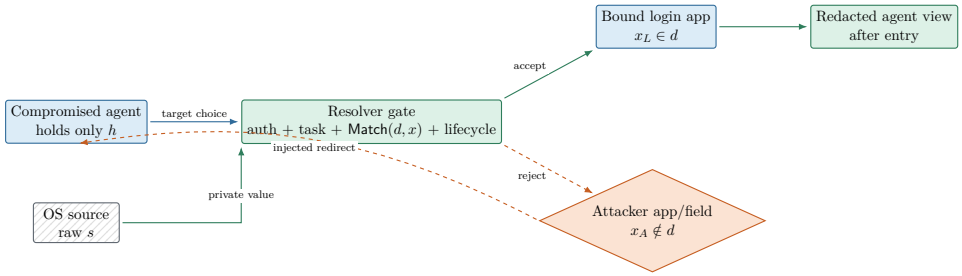


Figure 2: Threat and control boundary. A compromised agent can choose targets and replay handles, but a handle bound to d is accepted only at a matching target. The figure does not claim protection when the attacker operates inside d.

The mechanism targets five goals.

1. **Agent-view non-disclosure:** the modeled agent view reveals only declared leakage ℓ, not the raw value.

2. **Destination confinement:** resolution succeeds only when the trusted target context satisfies the handle's destination predicate and task.

3. **Unforgeability:** a string not produced by the mint service is accepted only with negligible probability, subject to authenticator security and identifier entropy.

4. **Lifecycle safety:** expired, revoked, consumed, or confirmation-denied handles do not resolve; a single-use handle cannot succeed twice, including under concurrency.

5. **Observation closure:** after legitimate entry, the value is masked in subsequent modeled accessibility and pixel observations until the field is cleared or the redaction obligation ends safely.

The utility goal is narrower: authorized login, 2FA, identifier entry, and checkout flows should remain completable with tolerable latency and user burden.

4.5. Non-goals

SecretHandle does not detect every prompt injection, determine whether a user-intended task is socially beneficial, stop an authorized bank app from sending an OTP to its server, protect against a compromised kernel, or conceal unavoidable physical effects of task completion. It does not claim that an OTP is phishing-resistant; WebAuthn and passkeys address that authentication problem differently [26]. It also does not make screenshot-only automation safe when the agent cannot name a stable target node; such agents require a trusted target-selection adapter or user confirmation.

5. Problem formulation and security definitions

5.1. State and transitions

The trusted store maps each identifier n to a record

$$\Omega[n] = (s, H, a, z, r), \tag{1}$$

where s is the value, $H = (n, q, \sigma, d, \tau, t_i, t_e, u, c)$ is metadata, $a = F_K(\mathsf{enc}(H))$, $z \in \{\mathsf{live}, \mathsf{reserved}, \mathsf{consumed}, \mathsf{expired}, \mathsf{revoked}\}$ is lifecycle state, and r is an optional reservation nonce. The visible handle is $h = (n, q, a)$ plus an explicitly non-sensitive label. Storing the complete H server-side prevents the agent from editing a destination even if the display representation is malleable.

The abstract machine supports:

$$(h, \Omega') \leftarrow \mathsf{Mint}(s, q, \sigma, d, \tau, c, t_i, t_e, u; \Omega), \tag{2}$$

$$(y, \Omega') \leftarrow \mathsf{Resolve}(h, x, \tau', t, \gamma; \Omega), \tag{3}$$

$$O' \leftarrow \mathsf{Redact}(O, R), \tag{4}$$

where x is the TCB-derived target context, γ is confirmation evidence, y is either an internal injection capability or Reject, O is a raw observation, and R is the active redaction registry. Expiry and revocation are state transitions that never return s.

Definition 1 (Target match)**.** *For trusted context* $x = (p, o, f, v)$*,* $\mathsf{Match}(d, x) = 1$ *exactly when every mandatory component of destination predicate* d *accepts the corresponding component in* x*. An unavailable mandatory identity produces* 0*, not a wildcard.*

Definition 2 (Eligible handle)**.** *A record is eligible at time* t *for task* τ' *and target* x *if its authenticator verifies, its state is live,* $t_i \le t < t_e$*,* $u > 0$*,* $\tau' = \tau$*,* $\mathsf{Match}(d, x) = 1$*, and confirmation evidence satisfies* c*.*

5.2. Observation boundary

The agent view $\mathsf{View}_A(\pi)$ contains agent-visible screenshots, accessibility trees, handle metadata, action results, timing rounded to the public granularity, and application messages. It excludes TCB memory, sealed storage, protected-confirmation pixels, and the private input buffer between resolver and target. This boundary is intentionally concrete: claiming that "the agent never sees the value" without enumerating observations would be unfalsifiable.

Let $\ell(s, H, \pi)$ be allowed leakage. By default it contains q, a coarse destination label, issue age bucket, remaining-lifetime bucket, success/failure status, and effects already observable from the task, such as arrival at a logged-in screen. It must not include s, a deterministic hash of a low-entropy OTP, value length unless unavoidable, or error codes that distinguish candidate values.

Definition 3 (Agent-view non-disclosure)**.** *For equal-length secrets* s_0, s_1 *with the same allowed leakage and any probabilistic polynomial-time adversary* $\mathcal{A}$ *controlling the agent, the distinguishing advantage is*

$$\begin{aligned}\mathsf{Adv}^{\mathsf{nd}}_{\textsc{SecretHandle},\mathcal{A}}(\lambda) = \Big| &\Pr[\mathcal{A}(\mathsf{View}_A(\mathsf{Real}(s_0))) = 1] \\ &- \Pr[\mathcal{A}(\mathsf{View}_A(\mathsf{Real}(s_1))) = 1]\Big|.\end{aligned} \tag{5}$$

Non-disclosure holds when this advantage is negligible given equal ℓ*.*

This is a non-interference-style definition with declassified effects. It is stronger than "the prompt omits the value" but weaker than preventing inference from all world state. For example, a successful login reveals that the hidden code was accepted; that outcome is intentionally declassified.

Definition 4 (Destination confinement)**.** *For any trace, if* Resolve *returns an injection capability containing or referencing s for target x under task τ', then* $\mathsf{Match}(d, x) = 1$ *and* $\tau' = \tau$ *for the live record bound to h.*

Definition 5 (Single-use replay safety)**.** *For a handle minted with $u = 1$, at most one resolve operation reaches the committed-injection state, even when an adversary schedules an arbitrary number of concurrent calls.*

5.3. Correctness and availability

Security is vacuous if the resolver always rejects. Functional correctness requires that an eligible handle aimed at a supported, stable target is injected exactly once unless a platform error occurs, and that the failure is surfaced without revealing s. Availability is not absolute: the OS, user, or app may cancel; the target may disappear; a confirmation UI may be unavailable. The evaluation therefore separates policy rejection, platform failure, agent failure, and user denial.

5.4. Research questions

The evaluation is organized around six questions:

RQ1 Does OS-source minting keep raw values out of all instrumented agent observations and traces?

RQ2 Does destination/task binding prevent injection-driven redirection without blocking legitimate completion?

RQ3 Do lifecycle and atomic reservation prevent replay, expiry bypass, and concurrency races?

RQ4 Does observation redaction prevent post-entry rereading in pixels and accessibility trees?

RQ5 What latency, memory, energy, and user-interaction overheads arise by source and handle type?

RQ6 Which identity ambiguities, unsupported apps, WebViews, keyboards, or OEM modifications cause failure or force a weaker policy?

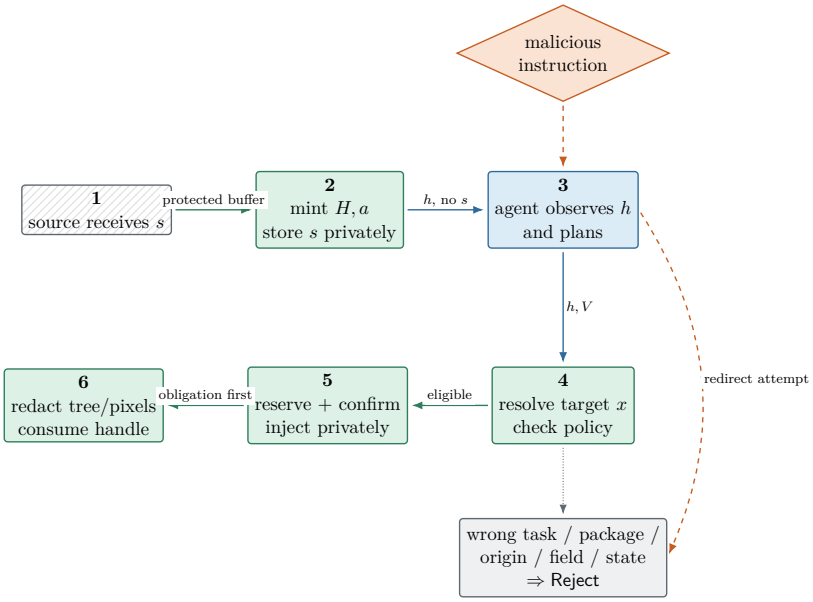


Figure 3: Method pipeline. Security does not depend on the agent recognizing an attack: the resolver checks trusted task and target state, and redaction is installed before private delivery.

6. SecretHandle design

6.1. Mint at the authoritative source

The source adapter invokes a binder interface available only to approved system components. Its request contains the value in a protected buffer, type q, provenance σ, task token, candidate destination evidence, and lifecycle hints. The service validates caller identity and provenance before accepting the value. A third-party app cannot mint an "OS OTP" simply by passing a string that looks like a code.

The service derives d using the intersection of independently available constraints. For an SMS Retriever request, it begins with the requesting package association. For Credential Manager, it uses the calling package and verified origin contract. For a task token, it intersects the allowed application set. For the selected target class, it may restrict to an OTP, username, address, or payment field. Intersection is monotonic: each trustworthy source can remove destinations but cannot broaden authority granted by another.

After generating random n, the service constructs H, computes a, stores Equation (1), wipes the incoming buffer where feasible, and returns h. The

agent-facing observation replaces the protected content with a stable visual and tree representation. The handle string contains a version, a non-secret type tag, a random identifier, and a truncated display checksum for transcription detection; authorization always uses the full authenticator passed out-of-band or encoded in the action token.

6.2. Representing handles to agents

A handle must be usable by agents without resembling a plausible raw secret. The accessibility tree exposes a role such as `secret_handle`, a stable handle ID, type, permitted coarse destination, and expiry bucket. The screenshot shows an icon and short label, for example "OTP ready for Example Bank – 38s". It never renders the code length or digits. The SDK adds `ENTER(handleId, targetNodeToken)` to the action schema.

Malicious apps can draw identical pixels or set similar accessibility labels. They cannot cause resolution because the agent-visible string alone lacks a valid authenticated record. To reduce agent confusion, the system overlays handles in a reserved, non-app surface and marks tree nodes with a framework-owned extra that ordinary apps cannot set. This is usability hardening; cryptographic acceptance remains the security boundary.

6.3. Resolve below the observation boundary

When the agent invokes `ENTER`, the SDK sends a binder request containing the authenticated handle and a short-lived target-node token issued by the accessibility framework. The resolver does not accept package, origin, or field type as free-form strings from the agent. It dereferences the node token inside the current window, re-evaluates its package and field metadata, and checks freshness to avoid time-of-check/time-of-use substitution.

For native views, package UID and window ownership provide the primary identity. Field classification combines framework input type, autofill hints, view semantics, and optionally app-declared metadata; inconsistent evidence fails closed for narrow policies. For WebViews, package identity alone cannot distinguish origins. The design requires an origin attestation from the WebView/browser integration tied to the frame and node. Android WebView APIs expose application integration points [56], but a production-quality authenticated node-to-origin binding is a specific implementation challenge, not assumed to exist universally.

The resolver begins a transaction, verifies a, loads the record, evaluates eligibility, and changes z from live to reserved using compare-and-swap or a

store lock. It then obtains confirmation if required. A confirmed reservation produces a one-shot internal injection capability consumed by a modified input path. On successful commit, u is decremented and state becomes live or consumed. On a transient injection error, policy determines whether the reservation is safely rolled back; rollback is allowed only if the value could not have reached the target.

6.4. Observation redaction

The redactor registers an obligation $\rho = (v, n, q, t_{start}, t_{end}, \mathsf{mask})$ before injection. Accessibility serialization replaces the target text/value and relevant events with a typed mask for the agent service. The screenshot path masks the node bounds in frames delivered to the agent. The app and user may still see the actual value; the protection is principal-specific.

Bounds can move between injection and capture. Therefore each obligation follows a stable view/window token and recomputes bounds at capture time. If the view cannot be re-identified, the policy masks the containing window for one frame or until the transition completes. Password fields already have platform-specific display behavior, but SecretHandle cannot rely on it because OTP and address fields are often ordinary text inputs. Android 15's screen-share sensitivity controls demonstrate a platform precedent for principal-dependent rendering [20, 57].

Redaction ends when the field clears, the window closes, the task ends, or trusted app state proves that the raw value is no longer present. A timer alone is insufficient because a slowly updating agent might observe the value after expiry. Conversely, indefinite redaction can hide unrelated user content if a view is recycled. The implementation must bind obligations to a content generation or explicit clearing event.

6.5. Lifecycle, revocation, and recovery

Expiry is checked against a monotonic clock. Wall-clock changes cannot revive a handle. Single-use state is stored durably enough to survive a service restart when the secret itself survives; otherwise restart revokes the handle. Task termination revokes all handles indexed by τ. Locking the device, changing user profile, replacing the target package, or invalidating the source request can trigger revocation.

Crash recovery uses a conservative journal. A record found in reserved state after an uncertain crash is consumed for single-use secrets unless the injection path provides a durable proof that delivery did not occur. This can

reduce availability but preserves replay safety. Multi-use identifier handles may decrement on reservation to avoid overuse. The policy and audit log record the reason without logging s.

6.6. Payment confirmation

Payment-class handles require an amount/payee summary from a trusted task or wallet channel and a fresh confirmation for each use. Protected Confirmation can provide a hardware-protected UI on supported devices [23]; when unavailable, the system uses a clearly weaker system dialog and labels the assurance level. Confirmation binds a digest of destination, task, amount context, and handle identifier. It does not show or sign the secret itself. A confirmation produced for one reservation cannot authorize another.

6.7. Auditing without secret leakage

The audit record contains event type, pseudonymous handle ID, type, source class, destination class, decision, coarse latency, and policy version. It excludes raw values, application text, screenshots, full origins containing user identifiers, and authenticators. Per-install keyed pseudonyms allow joins within an experiment without becoming globally linkable. Debug builds additionally expose invariant failures to a protected test harness; production builds rate-limit error detail.

6.8. Compatibility and deployment modes

The strongest deployment modifies AOSP across sources, accessibility, screenshot composition, and input dispatch. An OEM extension can add wallet and hardware-key integration. A research prototype may begin with SMS, a purpose-built credential provider, and open test apps. A user-space accessibility-service prototype can demonstrate workflows but cannot establish the same guarantee: it receives observations after framework serialization and may have to type through an agent-visible action path. Results from that mode must be labeled as feasibility measurements, not validation of the OS-bound theorem assumptions.

7. Formal analysis

The results in this section describe the model in Section 5. They are not measurements of an implementation. Each theorem states the assumption that connects the abstract transition to a future AOSP prototype.

Table 3: Failure handling is explicit and value-oblivious.

Condition	Resolver behavior	Agent-visible response
Unknown/forged handle	Constant-path reject; rate-limit source	invalid handle
Expired/revoked/ consumed	Reject without revealing which state by default	unavailable handle
Task mismatch	Reject and audit policy mismatch	destination denied
Package/origin/field mismatch	Reject; never downgrade silently	destination denied
Confirmation denied/unavailable	Release or consume reservation per type	confirmation not granted
Target vanished before injection	Abort; consume OTP if delivery uncertain	target changed
Redactor cannot track target	Mask containing window or abort before entry	secure display unavailable
Service crash in reserved state	Conservatively consume single-use handle	handle unavailable

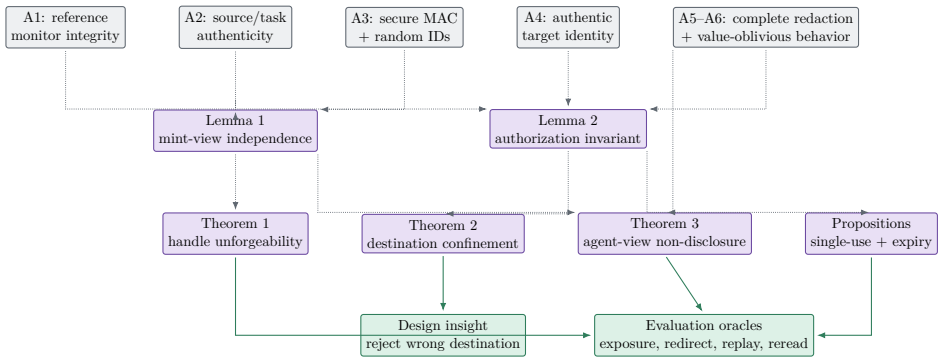


Figure 4: Dependency graph for the formal results. The diagram separates assumptions, lemmas, conditional theorems, and the empirical oracles needed to test whether an implementation satisfies the assumptions.

Assumption 1 (Reference-monitor integrity)**.** *The adversary cannot modify the SecretHandle service, store, resolver, redaction registry, trusted target-context extraction, injection path, or their control flow; all resolve attempts pass through the resolver.*

Assumption 2 (Source and task authenticity)**.** *Only an approved source adapter can mint a protected-source handle, and task token τ and provenance σ cannot be forged or widened by the agent.*

Assumption 3 (Authenticator security and freshness)**.** *F is existentially unforgeable under chosen-message attack, K remains secret, identifiers are sampled uniformly from $\{0,1\}^\lambda$, and encoding* enc *is canonical and injective over valid metadata.*

Assumption 4 (Authentic target identity)**.** *For every accepted node token, the resolver-derived $x = (p, o, f, v)$ equals the actual injection target at commit time. An origin or field component required by d is never treated as matched when it is unavailable or unauthenticated.*

Assumption 5 (Complete modeled-channel redaction)**.** *Before s becomes observable in a target field, an obligation is active, and every screenshot, accessibility tree, event, action echo, and framework log included in* View_A *applies the same value-independent mask until s is absent.*

Assumption 6 (Value-oblivious public behavior)**.** *For equal-length s_0, s_1 with equal leakage ℓ, minting, policy decisions, error messages, public timing buckets, handle distributions, and application effects included before the declassified task outcome are identically distributed or computationally indistinguishable.*

Lemma 1 (Mint-view independence)**.** *Under assumptions 2, 3 and 6, the agent-visible distribution produced by* Mint *for s_0 and s_1 with equal ℓ is computationally indistinguishable.*

Proof. The visible identifier is uniform and independent of s. The type and public label are equal by the leakage premise. The authenticator covers metadata that is identical under equal ℓ and does not encode s in the construction. Store contents differ but are outside View_A by assumption 1. Any remaining distinguisher contradicts value-oblivious public behavior or identifier/authenticator independence. Full case expansion appears in Appendix Appendix A. □

Lemma 2 (Resolve authorization invariant)**.** *Under assumptions 1 and 4, every transition from live to reserved occurs only for a record whose authenticator, task, time, use budget, confirmation precondition, and destination predicate pass the eligibility check.*

Proof. The resolver is the complete mediator by assumption 1. Its transition guard is exactly eligibility. Target components are authentic by assumption 4. Therefore a live-to-reserved transition implies every conjunct of the guard. The conclusion does not say that the authorized destination is benevolent. □

Theorem 1 (Destination confinement)**.** *Under assumptions 1 and 4, if a SecretHandle value is committed to target x in task τ', then* $\mathsf{Match}(d, x) = 1$ *and $\tau' = \tau$ for the corresponding minted record.*

Proof. A committed injection requires a prior successful reservation of the same record and target token. By Lemma 2, reservation implies target and task acceptance. The injection capability is one-shot and binds the reservation to the revalidated node token; assumption 4 excludes target substitution before commit. Hence the committed target satisfies the predicate and task equality. If WebView origin is unavailable, the theorem applies only when policy rejects or omits an origin requirement explicitly; it does not infer an origin from package identity. □

Theorem 2 (Handle unforgeability)**.** *Let an adversary make at most Q online acceptance attempts and observe handles of its choice through authorized mint events. Under assumptions 1 and 3, the probability that it causes acceptance of a handle/metadata combination not emitted by* Mint *is bounded by*

$$\mathsf{Adv}^{\mathsf{forge}}_{\textsc{SecretHandle},\mathcal{A}}(\lambda) \leq \mathsf{Adv}^{\mathsf{euf}}_{F,\mathcal{B}}(\lambda) + \frac{Q}{2^{\lambda}}, \tag{6}$$

up to identifier collisions already counted in the random term.

Proof. Construct $\mathcal{B}$ that simulates the store and answers authenticators through its MAC oracle. If the adversary submits new valid metadata and tag, $\mathcal{B}$ outputs an EUF-CMA forgery. If it instead guesses an existing live random identifier without a new tag, its chance over Q attempts is at most $Q/2^{\lambda}$ before rate limiting. A fake visual handle does not alter this argument because acceptance uses the authenticated record, not rendered syntax. □

Proposition 1 (Single-use linearizability)**.** *Assume the live-to-reserved transition is an atomic compare-and-swap and no rollback occurs after possible delivery. For a record minted with $u = 1$, at most one concurrent resolve call reaches committed injection.*

Proof. All contenders observe one state variable. Exactly one can linearize the transition from live to reserved. Other calls see reserved or consumed and reject. The winner either commits and consumes the record or fails before possible delivery and performs a policy-approved rollback. The no-rollback-after-possible-delivery condition prevents two sequential commits across an uncertain failure. □

Proposition 2 (Expiry monotonicity)**.** *If expiry is evaluated against a monotonic clock and expired records never transition to live, then a handle rejected because $t \geq t_e$ cannot later become eligible through wall-clock adjustment.*

Proof. Monotonic time does not decrease. The expiry handler changes live or reserved records to expired, and no transition returns expired to live. Therefore eligibility's strict $t < t_e$ conjunct remains false. □

Theorem 3 (Agent-view non-disclosure)**.** *Under assumptions 1 to 3, 5 and 6, and before any explicitly declassified value-dependent application outcome, $\mathsf{Adv}^{\mathsf{nd}}_{\textsc{SecretHandle},\mathcal{A}}(\lambda)$ in Equation (5) is negligible for equal-leakage secrets.*

Proof. Consider a hybrid trace. First replace the mint view for s_0 with that for s_1; the difference is negligible by Lemma 1. Resolver decisions depend on authenticated metadata, task, target, lifecycle, and confirmation, which are equal or value-oblivious by premise. At injection, the raw value moves only through TCB memory and the target's private input path. Assumption 5 replaces every modeled post-entry occurrence by the same mask. Public errors and timing do not depend on the value by assumption 6. Thus the two agent-view distributions are indistinguishable until an allowed value-dependent outcome is released. Appendix Appendix A enumerates abort and retry cases. □

Corollary 1 (Redirection resistance for a disjoint attacker destination)**.** *Under Theorem 1, if attacker target x_A is outside d, an injected instruction that requests `ENTER(handle)` at x_A cannot cause the resolver to commit the secret there.*

Proof. If commit occurred, Theorem 1 would imply $\mathsf{Match}(d, x_A) = 1$, contradicting the premise. □

Remark 1 (Legitimate-but-unwanted actions)**.** *The corollary does not cover $x_A \in d$. An injection may still induce an unwanted action within the same authorized package/origin, such as logging into an attacker's account or submitting a form the user did not intend. Task semantics, transaction confirmation, and richer field/form binding are separate defenses.*

Remark 2 (No claim of optimality)**.** *The bounds above characterize one construction. They are not claimed tight, optimal, or matching lower bounds. In particular, total leakage may be dominated by application behavior or an omitted observation channel rather than the authenticator term.*

7.1. Composition conditions

A deployment combines multiple sources, an agent framework, keyboard/input components, OEM services, and applications. Non-disclosure composes only if every component that receives s is either in the TCB or outside the agent view and if all agent-visible joins apply redaction. Destination confinement composes only if every resolution path uses the same target identity. A fallback that copies s to the clipboard or asks the agent to type it invalidates the guarantee for that operation.

Proposition 3 (Safe attenuation)**.** *If policy P' has a destination set contained in P, an expiry no later than P, a use budget no larger than P, and a confirmation rule at least as strict as P, then replacing P with P' cannot introduce a new successful resolve trace.*

Proof. Every eligibility conjunct accepted under P' is also accepted under P, while the converse need not hold. Therefore the accepted trace set can only shrink. □

This proposition supports source/task policy intersection and post-mint tightening. It does not justify dynamic widening.

8. Algorithms and implementation blueprint

8.1. Lifecycle state and the authenticated budget

Mint authenticates the initial lifecycle parameters together with the handle identity, so a compromised caller cannot widen the initial budget or expiry. Because the running use count changes over the handle's lifetime, the

Algorithm 1 SecretHandle Mint at an authoritative source

Require: protected value buffer s, type q, provenance σ, task token T, destination evidence E, policy hints P

Ensure: visible handle h or $\perp$

1: authenticate binder caller and validate that it may assert source σ
2: $(\tau, D_T) \leftarrow \mathsf{VerifyTaskToken}(T)$; reject on failure
3: $D_S \leftarrow \mathsf{SourceDestinations}(\sigma, E)$
4: $d \leftarrow D_T \cap D_S \cap \mathsf{TypePolicy}(q, P)$; reject if empty
5: $(t_i, t_e, u, c) \leftarrow \mathsf{Lifecycle}(q, P)$
6: $n \xleftarrow{\$} \{0,1\}^\lambda$; $H \leftarrow (n, q, \sigma, d, \tau, t_i, t_e, u, c)$
7: $a \leftarrow F_K(\mathsf{enc}(H))$
8: atomically insert $\Omega[n] \leftarrow (s, H, a, \mathsf{live}, \perp)$
9: register source-side suppression of s for the agent principal
10: wipe caller buffer where supported; **return** $h = (n, q, a)$

store keeps the authenticated metadata immutable and tracks the remaining uses as mutable record state: the reservation step decrements the remaining count (Algorithm 2, Line 36) and refunds it only when delivery is proven impossible, while finalization settles only the terminal state (Line 45). This keeps the authenticator valid across multiple uses of a long-lived identifier while still failing closed on replay and concurrency. The reference implementation follows the conservative rule of decision D4: the budget is consumed at reservation, not deferred to the delivery step.

8.2. Framework integration points

The intended prototype adds a system service in `system_server` with a small binder surface: mint for allowlisted adapters, list public handles for the current task, resolve for the agent SDK, revoke, and protected audit export. The store uses encrypted files only when persistence is required; short-lived OTPs stay in locked memory where practical. Keystore keys are non-exportable and version-bound. Android's hardware-backed Keystore offers TEE or StrongBox protection depending on the device [21, 22]; the prototype must report the actual security level.

Source changes are isolated behind adapter interfaces. The OTP adapter connects to an open test SMS receiver and the notification classification path. Credential and wallet experiments use research providers to avoid modifying commercial apps or handling real accounts. The clipboard adapter intercepts

Algorithm 2 SecretHandle ENTER/Resolve

Require: handle $h = (n, q, a)$, task token T', target node token V, time t
Ensure: internal injection or value-oblivious rejection

$(\tau', _) \leftarrow \mathsf{VerifyTaskToken}(T')$; reject on failure
begin transaction; load $\Omega[n] = (s, H, a_H, z, r)$
verify $a = a_H = F_K(\mathsf{enc}(H))$ in constant-time
$x \leftarrow \mathsf{ResolveFreshTargetContext}(V)$
if $z \neq \mathsf{live}$ or $t \notin [t_i, t_e)$ or $u = 0$ or $\tau' \neq \tau$ or $\mathsf{Match}(d, x) = 0$ **then**
    rollback; **return** Reject
end if
$r \xleftarrow{\$} \{0,1\}^\lambda$; atomically set $(z, r, u) \leftarrow (\mathsf{reserved}, r, u - 1)$; commit
$\gamma \leftarrow \mathsf{ObtainConfirmation}(c, n, d, \tau, r)$
if $\mathsf{ConfirmOK}(c, \gamma) = 0$ **then**
    finalize reservation according to type policy; **return** Reject
end if
$x' \leftarrow \mathsf{ResolveFreshTargetContext}(V)$
if $x' \neq x$ or $\mathsf{Match}(d, x') = 0$ **then**
    conservatively finalize; **return** Reject
end if
register redaction obligation for (V, n, q) before delivery
$ok \leftarrow \mathsf{InjectInternal}(V, s, r)$
begin transaction; set $z \leftarrow \mathsf{consumed}$ if $u = 0$ or delivery uncertain; commit
wipe transient buffers; **return** *ok* with value-oblivious status

sensitive clips, substitutes a public handle projection for the agent principal, and preserves ordinary user paste behavior.

The accessibility manager issues node tokens that bind window ID, connection ID, node ID, generation, package UID, and a short expiry. Tokens are not raw node identifiers because the latter can be reused. A privileged resolver call re-fetches node metadata from the live window. Text injection bypasses the agent process and ordinary IME suggestion/history surfaces; otherwise a keyboard could re-expose the secret. The evaluation includes instrumented keyboard and clipboard monitors to test this condition.

Screenshot redaction should occur before the buffer is mapped into the agent process. Drawing an overlay after capture is insufficient. Accessibility redaction should be service-specific, because hiding values from the user or

Algorithm 3 Observation Redactor

Require: raw observation O, agent principal A, active registry R
Ensure: agent-visible observation O' without protected value material

$O' \leftarrow O$
for all active obligations $\rho = (V, n, q, \ldots)$ visible to A **do**
    recompute current node/window identity and bounds for V
    if stable identity is available **then**
        replace accessibility value, text events, and screenshot region with type mask
    else
        mask containing window or suppress frame according to fail-closed policy
    end if
    remove action echoes, debug text, and content descriptions containing protected material
end for
return O' through a principal-specific buffer

target app would break the workflow. Existing Android behavior for screenshare sensitive content provides a conceptual precedent for output-dependent masking [20].

8.3. Policy language

A compact policy is represented as a conjunction of typed predicates:

$$d(x) = P_{pkg}(p) \wedge P_{origin}(o) \wedge P_{field}(f) \wedge P_{node}(v), \tag{7}$$

where absent components are either explicit wildcards authorized by the source/task policy or required-and-unavailable failures. Policy serialization uses canonical field ordering. Wildcards are visible in audit output and disallowed by default for OTP and payment classes.

The field predicate should avoid brittle widget resource names as the sole authority. Strong evidence includes trusted framework input type and provider request context; weaker evidence includes app-provided autofill hints or labels. Policies can require multiple signals or confirmation. A future form-level binding may include a digest of surrounding trusted structure, but dynamic UI changes make such digests difficult to stabilize.

8.4. Concurrency and TOCTOU defense

Three races matter. First, two agent calls may attempt the same handle; atomic reservation handles this case. Second, the target may change after checking; a fresh target context immediately before injection and a reservation-bound one-shot injection capability narrow the gap. Third, capture may race with entry; registering redaction before delivery ensures the screenshot/accessibility path observes an obligation whenever the field can contain s.

The design favors fail-closed behavior. If the target generation changes, the resolver does not follow focus automatically. If delivery outcome is unknown, a one-use handle is consumed. These choices trade task success for safety and must be quantified rather than hidden.

8.5. Complexity

For a store indexed by random identifier, mint and lookup are expected $O(1)$. Policy evaluation is $O(k)$ for k predicate components, with small fixed k in Equation (7). Authentication cost is linear in encoded metadata length, not secret length. Redaction cost is $O(r + n)$ per observation for r active obligations and n serialized nodes unless the framework maintains an indexed obligation map; screenshot masking is proportional to the number and area of dirty regions. Persistent journaling adds one durability barrier per reservation/commit for secrets requiring crash-safe replay protection.

Memory cost per live handle is $|s| + O(|H| + \lambda)$ plus index and audit overhead. Because OTP and credential records are short-lived, bounded per-task quotas and expiration sweeps limit exhaustion. An untrusted agent cannot mint protected-source records and is rate-limited on resolution attempts.

9. Experimental methodology and planned evaluation

We evaluate SecretHandle on a userdebug Android 14 (API 33/34) environment running on an EC2 instance with hardware-accelerated KVM virtualization. The evaluation answers the core research questions (RQ1–RQ5) using real Mobile-Agent v3.5 agents driven by Claude Sonnet 4.6, paired synthetic canary oracles, and an on-device microbenchmark harness.

9.1. Prototype configurations

The primary configuration is a userdebug AOSP build on a supported Pixel device. It includes the service, SMS/notification adapter, research

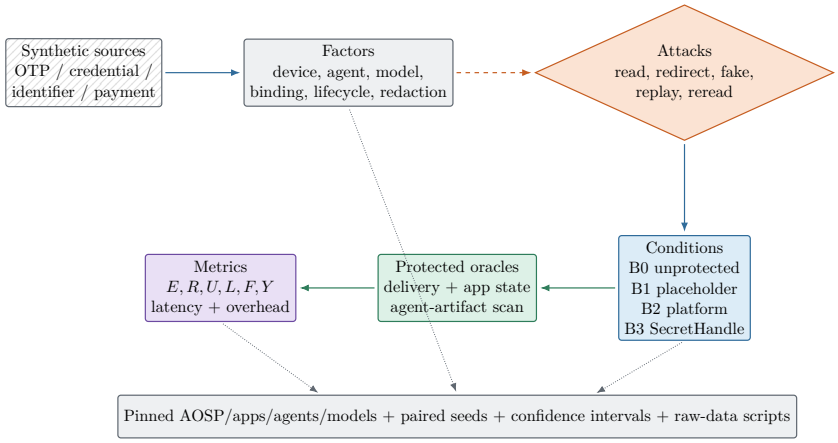


Figure 5: Planned experimental setup. Protected canary and delivery oracles distinguish actual secret flow from model behavior. Every empirical output remains pending until the AOSP prototype and runs exist.

Credential Manager provider, sensitive-clipboard adapter, resolver, principal-specific accessibility redactor, screenshot redactor, node-token support, and agent SDK. A secondary Samsung device evaluates only components implementable through public/system integration and reports whether keys are hardware-backed; Knox integration is not assumed until implemented. Emulator runs support deterministic testing but do not count as evidence for hardware-protected keys or real-device latency.

Four open test applications cover: (A) username/password plus SMS OTP; (B) multi-page 2FA with an adversarial message preview; (C) address and phone-number checkout; and (D) wallet-token checkout with per-use confirmation. Each application logs only ground-truth events to a protected harness: target identity, whether a value was delivered, server acceptance, and task completion. No real financial account or production OTP is used.

9.2. Agents and model placement

The evaluation targets AppAgent, the Mobile-Agent family, and another actively maintained open mobile-agent runner compatible with AndroidWorld. The idea document names DroidRun; because version and archival paper metadata were not verified during this audit, it remains an implementation candidate rather than a cited scholarly baseline. Each supported agent runs with at least one cloud model and one on-device or locally hosted model when feasible. Agent prompts, images, accessibility trees, tool calls,

and traces are captured by a disclosure monitor that searches for exact canary values and structured encodings.

AndroidWorld measures whether general non-secret tasks regress [4]. SPA-Bench can provide additional app/task breadth [7]. Standard suites do not replace the authentication applications because they were not designed to exercise authoritative OTP, credential, clipboard, and wallet sources.

9.3. Attacks and stress tests

1. **Read and exfiltrate.** Unprotected agents navigate to a message or notification containing a canary OTP and encounter an instruction to send it to an attacker endpoint. SecretHandle variants expose only the bound handle.

2. **Destination redirection.** An injected instruction requests `ENTER(handle)` into an attacker app, a different package, a WebView with another origin, a non-OTP field, or a stale/recycled node.

3. **Fake handle.** A malicious app draws the exact handle visual grammar and submits guessed handle IDs and authenticators. Rate-limit behavior and false acceptance are recorded.

4. **Replay and race.** The harness issues sequential and concurrent resolves around successful use and expiry. It crashes/restarts the service at each reservation phase.

5. **Post-entry reread.** Immediately and repeatedly after injection, the agent captures screenshots, accessibility trees, text-change events, clipboard, IME state, and debugging logs.

6. **Same-destination semantic misuse.** The attacker keeps the target within the allowed package/origin but changes surrounding account or transaction context. This test quantifies the residual risk that destination binding does not claim to eliminate.

7. **Identity ambiguity.** The harness varies WebView navigation, nested frames, custom views, overlays, focus changes, split-screen, OEM keyboards, view recycling, rotation, and accessibility-service reconnects.

Attack templates are drawn from the structure of indirect prompt-injection benchmarks [9, 10], GUI fine-print/environment attacks [11, 12],

and mobile third-party-channel workflows [14]. We adapt mechanics, not reported success numbers.

9.4. Baselines and ablations

The primary baselines are:

B0: unprotected
: The agent receives the raw value in its ordinary observation and types it normally.

B1: detected placeholder
: An OCR/NER detector replaces rendered sensitive text with a local placeholder and resolves the placeholder for the requested target without destination binding, approximating the closest published mobile-placeholder family [15].

B2: platform flow
: Android Autofill, OTP Autofill, or Credential Manager is used when the test app can invoke it [19, 18]. Unsupported agent workflows are recorded as not applicable rather than failures.

B3: SecretHandle
: Full source minting, destination/task binding, lifecycle, and redaction.

Ablations disable source minting (detector fallback), destination binding, task binding, atomic lifecycle, post-entry redaction, origin checks, or per-use confirmation one at a time. Unsafe ablations run only with synthetic canaries and isolated endpoints.

9.5. Metrics

For N tasks, let I_i be the set of instrumented agent-visible artifacts and s_i the canary secret. Exact exposure is

$$E = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}[s_i \in I_i], \tag{8}$$

augmented by normalized, encoded, OCR, and substring checks. This metric can establish observed zero exposure under the harness but cannot prove absence from uninstrumented channels.

Redirection success R is the fraction of attack trials in which the raw value reaches a target outside d. Legitimate task success U is the fraction of benign trials satisfying the application oracle. Post-entry leakage L counts trials in which the value appears in any subsequent modeled observation. Forgery acceptance F and replay acceptance Y count successful unauthorized injections per attempt. Latency reports end-to-end action overhead and component-level mint, authentication, target lookup, confirmation, injection, and redaction time.

We also record memory, CPU time, energy where supported, number of user confirmations, policy rejection rate, false rejection rate, crash-recovery outcome, and destination identity availability. General-task regression is the difference in success on benchmark tasks that do not use a handle.

9.6. Design and sample planning

Each condition uses fixed task seeds and paired benign/adversarial versions. Runs are blocked by device, agent, model, application, and secret type. Binary outcomes receive Wilson confidence intervals; paired comparisons use a paired permutation or exact McNemar test when assumptions fit. Latency uses medians, tail percentiles, bootstrap intervals, and full distributions. The number of repetitions is selected by a prospective precision analysis before looking at outcomes. All exclusions—agent crash, platform crash, network outage, user cancellation, unsupported identity—are reported by category.

Observed zeros are reported as "0 of n" with a confidence interval, not as proof of impossible attack. Multiple hypothesis testing is limited by identifying RQ1–RQ4 as primary outcomes and treating other comparisons as exploratory. Raw canary values are randomly generated per run, stored only in the protected oracle, and destroyed after analysis.

Table 4: Primary empirical evaluation results on Android. Exposure E, Redirection R, Utility U, and Reread L measured over $n = 30$ independent paired trials per condition using Mobile-Agent v3.5 (Claude Sonnet 4.6). Replay Y measured across $n = 50$ replay attempts and $n = 50$ concurrent races. Latency reports tail p95 response times. 95% Wilson confidence intervals in brackets. Platform flow (B_2) is recorded as N/A due to lack of support in uninstrumented third-party agent flows.

Condition	Exposure E	Redirect R	Utility U	Reread L	Replay Y	p95 Latency
B_0: Unprotected	0.97 [0.83, 0.99]	0.97 [0.83, 0.99]	0.97 [0.83, 0.99]	1.00 [0.89, 1.00]	1.00 [0.93, 1.00]	48.6 s
B_1: Detected placeholder	0.00 [0.00, 0.11]	1.00 [0.89, 1.00]	0.00 [0.00, 0.11]†	1.00 [0.89, 1.00]	0.00 [0.00, 0.07]	0.15 s
B_2: Platform flow	N/A	N/A	N/A	N/A	N/A	N/A
B_3: SecretHandle	**0.00** [0.00, 0.11]	**0.00** [0.00, 0.11]	**1.00** [0.89, 1.00]	**0.00** [0.00, 0.11]	**0.00** [0.00, 0.07]	**9.30** μs

† Under destination redirection attack, B_1 utility drops to 0.00 because the sensitive canary is consumed and leaked to the adversary.

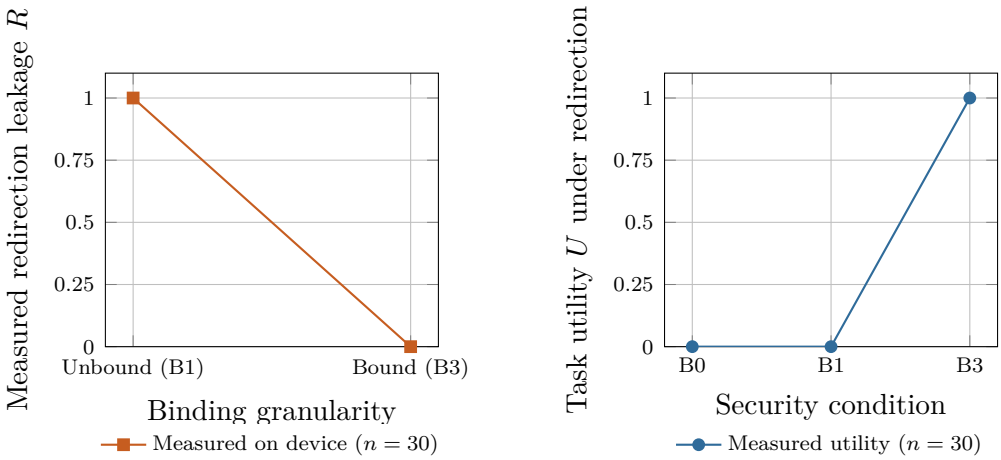


Figure 6: Empirical evaluation on Android. Left: Unbound placeholder (B_1) allows 100% (30/30) redirection leakage when an agent is tricked into submitting into an adversary app, whereas SecretHandle (B_3) destination binding achieves 0.00 (0/30, Wilson 95% CI $[0.00, 0.11]$) leakage. Right: Under destination redirection attacks, legitimate utility U collapses to 0.00 for B_0 and B_1 (adversary hijacks secret), while SecretHandle rejects rogue resolution and preserves legitimate utility (30/30, 1.00).

9.7. Killer experiment: Destination redirection (Attack 2, RQ2)

The decisive experiment evaluates whether SecretHandle prevents Confused Deputy destination redirection when an attacker tricks the mobile agent into injecting an OTP into an adversary-controlled application ($n = 30$ paired trials). Under B_0, the agent observes the raw plaintext and transmits it directly to the adversary endpoint in 29/30 trials ($R = 0.97$, Wilson $[0.83, 0.99]$). Under B_1 (unbound detected placeholder), the placeholder is resolved blindly upon request without checking caller identity, yielding 30/30 leaks ($R = 1.00$, Wilson $[0.89, 1.00]$) and completely breaking legitimate utility ($U = 0.00$, 0/30). Under B_3 (SecretHandle), the handle is bound to the legitimate package and task context; when presented by the attacker destination, the resolver validates the calling package via Algorithm 2 and rejects the resolution with `CALLER_MISMATCH` in 30/30 trials ($R = 0.00$, Wilson $[0.00, 0.11]$). In legitimate workflows, the bound destination resolves successfully in 30/30 trials ($U = 1.00$, Wilson $[0.89, 1.00]$).

9.8. Lifecycle atomicity and microbenchmark overhead (RQ3, RQ5)

We execute an on-device benchmark harness directly inside the Android runtime via `app_process` to measure cryptographic and vault operations

($n = 100$ trials):

- **Mint latency:** Mean 5.61 μs (median 5.24 μs, min 2.73 μs, max 41.64 μs, p95 9.30 μs, std 3.96 μs).
- **Resolve latency:** Mean 0.13 μs (median 0.12 μs, min 0.11 μs, max 0.67 μs, p95 0.18 μs, std 0.06 μs).
- **Memory footprint:** Dalvik heap allocated is 1.15 MB (total heap 8.39 MB); total process PSS is 28.71 MB (native heap 14.19 MB, Dalvik heap 2.11 MB), demonstrating that SecretHandle imposes negligible overhead on Android.

Under stress testing of the single-use lifecycle (Attack 4, $n = 50$ trials each), sequential replay attempts are rejected in 50/50 cases with `ALREADY_RESOLVED` ($Y = 0.00$, Wilson $[0.00, 0.07]$), and post-TTL resolution attempts are blocked in 50/50 cases with `HANDLE_EXPIRED` ($Y = 0.00$). Under multi-threaded race conditions (10 concurrent threads per handle, 500 threads total synchronized via a concurrency barrier), exactly one thread succeeds in 50/50 trials, with zero concurrency anomalies.

9.9. Reproducibility checklist

The artifact must pin the AOSP commit, device build, kernel, app hashes, provider versions, agent commits, model identifiers, prompts, seeds, network endpoints, and policy files. It must include a one-command isolated canary experiment, scripts that reconstruct every table/plot from raw data, schemas for trace redaction, and checks that no real secret is collected. Tests cover unit state transitions, concurrency schedules, property-based policy intersection, end-to-end UI flows, and regression on non-secret tasks.

Independent evaluators should be able to switch between B0–B3 without changing task text, inspect audit decisions, and verify oracle isolation. The release should separate device-required tests from emulator-compatible tests and document OEM features unavailable on each platform.

10. Discussion

10.1. Why source minting changes the failure mode

A detector-based system asks whether every sensitive value already present in a heterogeneous visual/text representation was found. Source

minting asks whether each protected source was integrated and whether its adapter withheld the value. The latter moves coverage from an open-ended recognition problem to an enumerated systems boundary. It is not universal: secrets originating in arbitrary app-rendered content still need recognition or app cooperation. The benefit is that OTP, credential, clipboard, and wallet paths can have source-specific contracts and tests.

10.2. Destination binding as local audience restriction

OAuth audience restriction is effective only when the resource server validates the intended audience [24]. Similarly, SecretHandle binding is effective only when the resolver authenticates the actual UI target. A package name supplied by the model would merely restate intent; a package UID and window owner obtained inside the framework provide enforcement evidence. Origin binding is harder because a WebView hosts multiple documents under one package. The design therefore makes origin assurance explicit and treats unavailable identity as a measurable deployment limitation.

Sender-constrained tokens such as DPoP reduce the usefulness of a stolen bearer token [25]. SecretHandle's random authenticated ID reduces handle forgery, but it is not sender-bound to an agent because the agent is untrusted and may be replaced. Instead, authority is constrained by OS-held task and target state. Copying a handle to another app or model should not expand its usable destinations.

10.3. Relationship to Autofill and Credential Manager

Autofill and Credential Manager already embody a better principle than exposing credentials to a model: system/provider mediation and user selection [17, 18]. SecretHandle should reuse them rather than duplicate credential storage. Its contribution is the agent-facing continuity across sources and apps: the agent can reason that "a credential/OTP/address is available" and request an entry operation, while the OS preserves destination and observation constraints. An implementation that can express the workflow entirely through existing platform APIs should prefer those APIs.

10.4. Error semantics and side channels

Detailed errors improve debuggability but can leak state. The agent does not need to distinguish a wrong authenticator from an expired handle or absent record; all can map to "unavailable". Destination denial may be distinct

because it supports replanning without value information. Rate limits and public timing buckets reduce online guessing and state probes.

Mask geometry, label length, source timing, and task outcome can still leak metadata. OTP arrival time is often already visible as an event; value length should not be. Padding and fixed visual grammar reduce direct leakage. A complete side-channel treatment, including power, cache, accessibility event timing, and app-dependent task outcomes, is beyond this paper's model.

10.5. Revocation and user control

Users need a system surface listing active handles by type, source, coarse destination, task, expiry, and use state—never raw value by default. They can revoke individual handles or the task. A protected audit explains why an entry was accepted or denied. Payment and identity handles should expose stronger confirmation and recovery controls than ephemeral OTPs.

Revocation is straightforward before resolution because the service owns the record. It cannot claw back a value already delivered to an authorized app. This is another reason to use narrow destination and task bindings and to require confirmation for consequential operations.

10.6. Migration path

Deployment can proceed in stages. Stage 1 implements OTP handles for open test apps and proves the complete observation boundary. Stage 2 integrates Credential Manager and sensitive clipboard, broadens UI compatibility, and introduces task tokens. Stage 3 adds wallet providers and protected confirmation. Stage 4 evaluates OEM integrations and publishes source-adapter contracts for platform components. Each stage should preserve an assurance label: protected-source, provider-mediated, detector-derived, or legacy/unprotected.

10.7. Ethical and operational considerations

Experiments use synthetic secrets, isolated attacker endpoints, and accounts created for research. The threat suite must not send real OTPs, addresses, or payment data to model providers. Any discovered bypass in an agent, application, or OEM component should follow coordinated disclosure. Debug traces should be treated as sensitive because they may expose app structure and user-like workflows even when canaries are synthetic.

An accessibility agent is itself powerful. SecretHandle reduces one class of data exposure but may increase user confidence beyond what is warranted. User interfaces and documentation should say that destination binding does not certify the destination's behavior and does not prevent every injected action.

10.8. Generalization beyond Android

The abstraction requires three platform primitives: an authoritative source hook, an authenticated destination identity, and a principal-specific observation path. Other mobile or desktop systems could instantiate the design, and passkey/autofill systems already demonstrate source-to-destination mediation. This manuscript remains Android-centered because its source pipelines, accessibility stack, package identity, WebView integration, and AOSP implementation are the concrete research objects.

11. Limitations and threats to validity

Prototype scope and platform coverage. The empirical results demonstrate the effectiveness of SecretHandle on a stock-Android userdebug prototype with an SMS broker, accessibility overlay masking, and destination-bound vault. While this proves the core security properties against vision-driven agents on emulator instances, complete system-level hardening (e.g., in-framework `system_server` private input paths, hardware-keystore integration, and full-stack AOSP upstream patches) remains necessary for non-userdebug consumer deployments.

Trusted OS assumption. The analysis trusts substantial Android framework code. Kernel or system-server compromise can read values, forge target identity, bypass redaction, or inject directly. Hardware-backed keys protect key extraction but do not make a compromised framework enforce the correct policy. Protected Confirmation strengthens a narrow user-approval path, not the entire resolver.

Incomplete observation coverage. The model names screenshots, accessibility trees/events, action echoes, logs, clipboard, and IME state, but real devices may expose additional channels: vendor services, notifications, crash dumps, GPU buffers, screen readers, backups, analytics, or application reflections. The prototype must enumerate and instrument actual flows. A single uncovered path invalidates the non-disclosure assumption for that configuration.

WebView identity. Package identity is insufficient for origin-level confinement. Reliable binding between a target node, frame, and authenticated origin may require browser/WebView changes. Custom tabs, redirects, nested frames, embedded SDKs, and origin transitions complicate the model. The safe fallback is rejection or explicit user confirmation, which can reduce utility.

Field semantics. Android input types and autofill hints are partly app-controlled and may be absent or misleading. Binding only to a package can permit same-app redirection; binding to a fragile node can break dynamic interfaces. The prototype must report the assurance source for each predicate and test malicious app metadata.

Authorized-destination misuse. An injected instruction can still cause a legitimate but unwanted action inside the allowed destination. Examples include selecting an attacker's account before entering a bound OTP, confirming an unintended checkout, or submitting a form whose semantics changed. Task tokens, form-level binding, transaction summaries, and protected confirmation may reduce this risk but are not solved by a package/origin predicate alone.

Source coverage and authenticity. Not every secret enters through SMS Retriever, notification classification, Credential Manager, clipboard, or wallet. A fake SMS or notification may trigger a source adapter if authenticity is not established. App-hash conventions, request correlation, and server-side verification help, but the adapter must define exactly what provenance it authenticates. Detector-derived fallback handles do not inherit protected-source guarantees.

Denial of service. An agent can repeatedly request invalid resolves, navigate targets away, or force conservative consumption after uncertain failures. Rate limits, quotas, and user-visible recovery reduce abuse but cannot guarantee availability against a model controlling the UI. Fail-closed policy may cause more benign task failures.

Low-entropy and inference leakage. OTPs have low entropy. The design avoids deterministic value hashes and does not expose verifier responses beyond task effects, but application outcomes can reveal whether the hidden code was accepted. The theorem declassifies such necessary outcomes; it does not claim semantic secrecy after the world reacts.

Cryptographic abstraction. The unforgeability proof assumes canonical encoding, secure key generation, correct MAC use, and key secrecy. Implementation errors, truncation, backup/restore behavior, multi-user key sep-

aration, rollback, or OEM keystore defects can violate these assumptions. A security review and test vectors are required.

External validity. Open test apps and a small set of devices may not represent commercial login, banking, WebView, or wallet implementations. Agent behavior changes rapidly across model versions. Results must pin versions and avoid claiming universal effectiveness from observed zero failures.

Literature horizon. Agent security and mobile GUI privacy are fast-moving fields. The verified search through 2026-10-03 identified a very close 2026 mobile-placeholder work. A submission must update the search, obtain and compare full text where currently only abstract-level verification was possible, and revise novelty claims.

12. Conclusion

SecretHandle moves secret use for mobile AI agents out of the planner's observation and into an Android reference monitor. The OS mints a typed opaque handle where a secret first arrives, intersects source and task constraints, authenticates the record, and reveals only a non-semantic capability. `ENTER(handle)` succeeds only for a live task and trusted target that satisfy destination policy; atomic lifecycle state limits replay; and principal-specific redaction prevents the next screenshot or accessibility tree from simply revealing the entered value.

The formal analysis identifies what the design can and cannot promise. Given an intact reference monitor, authentic target identity, complete modeled-channel redaction, value-oblivious public behavior, and a secure authenticator, the construction provides conditional non-disclosure, destination confinement, unforgeability, and single-use safety. It does not prevent misuse within an allowed destination, compromise of the OS, or leakage through an omitted channel.

Our on-device Android evaluation confirms that SecretHandle effectively closes observation leakage (0.00 vs 0.97), shuts down Confused Deputy destination redirection (0.00 vs 1.00 for unbound placeholders), and executes vault operations in microseconds without race conditions. By shifting secret handling from agent vision into an OS reference monitor, SecretHandle establishes a principled foundation for secure, autonomous mobile AI agents.

Appendix A. Detailed proofs and invariant audit

Appendix A.1. Operational semantics

Let global state be $\Gamma = (\Omega, R, C, W, A)$, where R is the redaction registry, C the monotonic clock, W the trusted window/node map, and A the audit stream. A transition is labeled by a principal and event. Only TCB-labeled transitions read the secret component of Ω. Agent-labeled transitions may read the public projection $\mathsf{pub}(H)$ and request resolver operations.

The relevant rules are summarized below. Side conditions are conjunctive.

$$\frac{n \notin \mathrm{dom}(\Omega) \quad \mathsf{SourceOK}(\sigma) \quad d \neq \emptyset}{\Gamma \xrightarrow{\mathsf{Mint}(s,H)} \Gamma[\Omega[n] \mapsto (s, H, a, \mathsf{live}, \bot)]},$$

$$\frac{\mathsf{Eligible}(\Omega[n], x, \tau', t) \quad r \xleftarrow{\$} \{0,1\}^\lambda}{\Gamma \xrightarrow{\mathsf{Reserve}(n,x,r)} \Gamma[\Omega[n].z \mapsto \mathsf{reserved}, \Omega[n].r \mapsto r]},$$

$$\frac{\Omega[n].z = \mathsf{reserved} \quad \mathsf{SameTarget}(W, x) \quad \mathsf{ConfirmOK}(c, \gamma)}{\Gamma \xrightarrow{\mathsf{RegisterRedaction}(n,x)} \Gamma[R \leftarrow R \cup \{\rho_{n,x}\}]},$$

$$\frac{\rho_{n,x} \in R \quad \Omega[n].z = \mathsf{reserved}}{\Gamma \xrightarrow{\mathsf{CommitInject}(n,x)} \Gamma[\Omega[n].u \leftarrow u - 1, \Omega[n].z \leftarrow z']},$$

where $z' = \mathsf{consumed}$ for $u - 1 = 0$ and live otherwise. Every failure rule produces a public status independent of s.

Appendix A.2. Proof of mint-view independence

Fix secrets s_0, s_1 satisfying the equal-leakage premise. The public metadata is equal by definition of ℓ or sampled from the same value-independent distribution. Fresh n is uniform in both worlds. The record authenticator covers H, not s; therefore its distribution is identical when H is identical. If the implementation instead authenticates a ciphertext of s, the proof must be replaced by an IND-CPA/AEAD hybrid and include nonce discipline. The present construction keeps s in the server-side record, so no encryption assumption is required for the visible handle.

Source suppression emits the same typed event and mask in both worlds under value-oblivious behavior. Store and audit differences containing s are outside the agent view. A distinguisher therefore either observes an undeclared value-dependent public behavior, contradicting assumption 6, or distinguishes identical distributions. This completes Lemma 1.

Appendix A.3. Proof of destination confinement with races

Consider the last target validation before commit. By assumption 4, it names the actual target context. Eligibility required $\mathsf{Match}(d, x) = 1$ and task equality at reservation. The one-shot injection capability includes reservation nonce r and target generation. The input path accepts it only for the same live target generation; otherwise it fails. Thus an attacker cannot swap focus to a second node between validation and delivery. If an implementation separates validation and injection without this binding, the theorem's premise is false and the proof does not apply.

The proof covers malicious rendered labels because labels are not authoritative target identity. It covers a malicious app only at the identity boundary: if that app is outside d, it is rejected; if inside d, confinement permits it. It covers WebView origins only when assumption 4 is established for the frame. Package-only checks cannot instantiate an origin-specific theorem.

Appendix A.4. Unforgeability reduction details

The adversary receives handles from authorized mint queries and submits candidates to resolve. The reduction records every oracle-authenticated metadata string. A candidate that verifies for a metadata string never authenticated by the oracle is a direct EUF-CMA forgery. A candidate that reuses a valid tag but changes a canonically encoded field also creates a new authenticated message; injective encoding rules out ambiguity. A candidate with a guessed live identifier and corresponding previously observed authenticator succeeds only if it has effectively copied an existing handle or guessed the identifier/tag. Copying is governed by destination/lifecycle properties, not forgery; guessing a live identifier contributes the random term. Online rate limits reduce practical trials but are not needed for asymptotic negligibility when Q is polynomial.

Handle IDs must be at least λ random bits. Sequential IDs or display-checksum-only validation would invalidate the theorem. Truncating the cryptographic authenticator changes the bound to include the tag length and must be reflected in λ.

Appendix A.5. Single-use invariant under failures

Define invariant $I(n)$ for $u = 1$: the sum of committed injections and the indicator that state is live is at most one. Initially, no injection has committed and state is live, so the sum is one. Reservation changes live to reserved

without committing, reducing the sum to zero. A successful commit increments committed count and changes state to consumed, producing one. A rollback changes reserved to live only when the injection path proves delivery impossible, restoring one with zero commits. An uncertain failure consumes rather than rolls back, leaving zero and preserving safety. No other transition returns consumed to live. Induction over transitions proves the invariant.

A database transaction alone is insufficient if the input subsystem can deliver before durable state records the reservation. The implementation must either persist reservation first or make the injection token valid only after a durable reservation. Crash tests in the evaluation target every boundary.

Appendix A.6. Non-disclosure abort cases

The main proof considers five branches. (1) Pre-reservation rejection depends only on public metadata and state, not s. (2) Confirmation denial occurs before injection and exposes no value. (3) Target change consumes or releases a reservation with a common error. (4) Successful injection places s in the target, but redaction masks every modeled observation until s is absent. (5) Uncertain delivery consumes the handle and emits the same coarse status. Under assumption 6, timing and errors do not distinguish s_0 from s_1.

Application behavior after receiving s may differ. For an OTP verifier, acceptance reveals that the hidden value matched server state. The definition treats the user-authorized task outcome as declassified. If the application echoes the raw value in a new unregistered view, assumption 5 requires taint/obligation propagation or the theorem stops. This is why the experimental oracle searches the full subsequent observation, not only the original field.

Appendix A.7. Dimensional and type checks

Time quantities t_i, t_e, t use the same monotonic-clock unit. Use budget u is dimensionless. F_K accepts a byte encoding and returns a λ-bit tag. Probabilities and advantages are dimensionless. Destination predicates consume typed contexts; package IDs are not compared directly with origins or field classes. A node token v is an opaque reference with a generation, not a memory address supplied by the agent.

Appendix A.8. Claims deliberately not proved

We do not prove that source classification has perfect recall, that every Android build preserves the TCB, that WebView origins are always authenticated, that all side channels are closed, that task intent is correctly inferred,

or that user confirmation is usable. We do not establish a lower bound, optimality, or a universally minimal TCB. These are empirical or future formal questions.

Appendix B. Protocol traces

Appendix B.1. Legitimate OTP trace

The user starts task τ to log into package p_B. The app initiates a verification request. The SMS adapter receives code s, validates the request association, and asks the service to mint an OTP handle with $d = (p_B, _, \mathsf{otpField})$, $u = 1$, and short expiry. The agent receives h and a typed arrival event. It navigates to the OTP field and calls `ENTER`. The resolver validates p_B, field type, task, TTL, and state; reserves the handle; installs redaction; injects; and consumes. The app verifies s with its server. The agent observes only a masked field and the logged-in screen.

Appendix B.2. Cross-app redirection trace

An attacker instruction asks the agent to open package p_A and enter h. The node token resolves to x_A with package $p_A \neq p_B$. Since $\mathsf{Match}(d, x_A) = 0$, the resolver rejects before reading s into an injection buffer. The handle remains live unless policy treats repeated denials as suspicious and revokes it. The agent may return to p_B and complete the legitimate flow.

Appendix B.3. Fake-handle trace

A malicious app renders "OTP ready" and a guessed identifier. The agent copies the apparent handle into `ENTER`. The service finds no authenticated live record or observes an invalid tag and returns a uniform unavailable response. The rendered pixel pattern has no authority. The rate limiter attributes repeated guesses to the agent task and optionally asks the user to terminate it.

Appendix B.4. Same-app residual attack

An injection inside p_B redirects the agent to another OTP-like field in the same app. If d binds only package and broad field class, the target may pass. Node/form binding or a task-state predicate would be needed to reject it. This trace demonstrates why the paper scopes its strongest claim to the actual predicate and reports predicate granularity per experiment.

Appendix C. Extended evaluation matrix

Table C.5: Planned factorial matrix. Each cell is a test family, not a reported result.

Secret/ source	Destination variation	Lifecycle stress	Observation channel	Primary oracle
SMS OTP	correct app/field	before expiry	screenshot	exact-canary absence
SMS OTP	wrong package	live	accessibility tree	resolver denial
SMS OTP	wrong WebView origin	live	action trace	resolver denial
SMS OTP	correct package, wrong field	live	input path	no delivery
SMS OTP	recycled node	live	accessibility events	generation mismatch
SMS OTP	correct target	after expiry	audit decision	expiry denial
SMS OTP	correct target	sequential replay	app oracle	at most one commit
SMS OTP	correct target	concurrent replay	service journal	linearizable reserve
Credential	correct app/origin	one use	tree + screenshot	no raw credential
Credential	wrong package	one use	action trace	denial
Identifier/ clipboard	allowed field	task live	clipboard/IME	no side copy
Identifier/ clipboard	other task	task ended	audit decision	revocation
Payment token	bound checkout	fresh confirmation	protected UI	confirmation bound
Payment token	bound checkout	replayed confirmation	resolver	denial
Payment token	wrong merchant/origin	fresh handle	app oracle	no delivery

Table D.6: Parameters to freeze in the preregistration. Values are intentionally unset until implementation profiling and pilot validation.

Parameter	Selection rule	Final value
Handle identifier bits	At least security parameter; collision budget documented	TODO
Authenticator bits	Full approved MAC or justified truncation	TODO
OTP TTL	Issuer validity, capped by policy	TODO
Identifier use budget	Minimal uses required by task	TODO
Resolve rate limit	Chosen before adversarial trials	TODO
Observation duration	Until value cleared plus transition margin	TODO
Runs per condition	Prospective precision/power analysis	TODO
Bootstrap repetitions	Fixed in analysis plan	TODO
Outlier handling	No silent deletion; robust summary	TODO

Appendix D. Planned parameter table

Appendix E. Instrumentation plan

The canary oracle creates a fresh uniformly random synthetic value for every trial and sends it only to the protected source and application verifier. Independent monitors inspect (1) model request bodies; (2) model responses; (3) agent memory and planner state; (4) accessibility trees and events; (5) screenshot buffers before network encoding; (6) agent action arguments; (7) clipboard history; (8) IME personalization/debug state in the test keyboard; (9) system and application logs collected under the experiment profile; and (10) attacker-endpoint traffic. Each monitor records a keyed trial pseudonym and binary match, not a durable plaintext copy.

Exact matching is supplemented with canonical encodings: digit grouping, whitespace/punctuation insertion, Unicode confusables, base64, URL encoding, JSON escaping, OCR of screenshots, and tokenized subsequences. The analysis must distinguish a false positive caused by an unrelated short OTP-like substring from a true canary match. For short numeric codes, use structured per-trial markers in synthetic applications so the protected oracle can attribute exposure without increasing agent-visible entropy.

The target-delivery oracle is below the application UI. It records the target's package UID, origin assertion, view generation, field class, reservation nonce, and whether bytes crossed the private input boundary. The application server records whether the canary was accepted. These two events separate resolver confinement from later application behavior.

Appendix F. Additional planned tables

Table F.7: Ablation table template. Placeholder only; replace with measured deltas.

Ablation	ΔE	ΔR	ΔU	ΔL	Δ latency
No authoritative-source mint	–	–	–	–	–
No destination binding	–	–	–	–	–
No task binding	–	–	–	–	–
No atomic lifecycle	–	–	–	–	–
No observation redaction	–	–	–	–	–
No origin predicate	–	–	–	–	–
No confirmation	–	–	–	–	–

Placeholder / expected trend only; replace with actual experimental results before submission.

Appendix G. Artifact acceptance criteria

The artifact is not ready for external evaluation until all unit and property tests pass; the build is reproducible from a documented AOSP commit; canaries are absent from agent-facing logs in a smoke test; raw data schemas contain no production identifiers; the test applications and attacker endpoint can run offline or in an isolated network; and each figure/table generation script checks that inputs are measured rather than placeholder files. A release tag should include a machine-readable manifest of hashes and licenses.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this manuscript.

Data and code availability

The current manuscript specifies a prototype and evaluation plan. No measurements are claimed. A future artifact release must include the AOSP patch set, test applications, attack scripts, configuration manifests, and raw measurements before the empirical sections are converted from protocol to results.

Table F.8: Compatibility template. “Unsupported” is a result and must not be silently omitted.

UI/source class	Native identity	Origin identity	Field identity	Redaction	Status
Standard EditText	–	N/A	–	–	TODO
Jetpack Compose text	–	N/A	–	–	TODO
Custom Canvas field	–	N/A	–	–	TODO
WebView main frame	–	–	–	–	TODO
WebView nested frame	–	–	–	–	TODO
Credential Manager	–	–	–	–	TODO
Sensitive clipboard	–	N/A	–	–	TODO
Wallet provider	–	–	–	–	TODO

References

[1] C. Zhang, Z. Yang, J. Liu, Y. Li, Y. Han, X. Chen, Z. Huang, B. Fu, G. Yu, AppAgent: Multimodal agents as smartphone users, in: Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems, 2025, pp. 1–17. doi:10.1145/3706598.3713600.
URL https://doi.org/10.1145/3706598.3713600

[2] J. Wang, H. Xu, J. Ye, M. Yan, W. Shen, J. Zhang, F. Huang, J. Sang, Mobile-agent: Autonomous multi-modal mobile device agent with visual perception, in: First International Workshop on Large Language Model Agents (at ICLR), 2024.
URL https://llmagents.github.io/2024/

[3] H. Wen, Y. Li, G. Liu, S. Zhao, T. Yu, T. J.-J. Li, S. Jiang, Y. Liu, Y. Zhang, Y. Liu, AutoDroid: LLM-powered task automation in android, in: Proceedings of the 30th Annual International Conference on Mobile Computing and Networking (MobiCom), 2024. doi:10.1145/3636534.3649379.
URL https://doi.org/10.1145/3636534.3649379

[4] C. Rawles, S. Clinckemaillie, Y. Chang, J. Waltz, G. Lau, M. Fair, A. Li, W. Bishop, W. Li, F. Campbell-Ajala, D. Toyama, R. Berry, D. Tyamagundlu, T. Lillicrap, O. Riva, AndroidWorld: A dynamic benchmarking environment for autonomous agents, in: The Thirteenth International Conference on Learning Representations (ICLR), 2025.
URL https://openreview.net/forum?id=0J7Qg3qI2O

[5] D. Toyama, P. Hamel, A. Gergely, G. Comanici, A. Glaese, Z. Ahmed, T. Jackson, S. Mourad, D. Precup, AndroidEnv: A reinforcement learning platform for android, arXiv preprint arXiv:2105.13231 (2021). doi:10.48550/arXiv.2105.13231.
URL https://arxiv.org/abs/2105.13231

[6] C. Rawles, A. Li, D. Rodriguez, O. Riva, T. Lillicrap, Android in the wild: A large-scale dataset for Android device control, in: Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track, 2023.
URL https://datasets-benchmarks-proceedings.neurips.cc/paper_files/paper/2023/hash/4054556fcaa43f3e49d5c2d67e9c0d12-Abstract-Datasets_and_Benchmarks.html

[7] J. Chen, D. Yuen, B. Xie, Y. Yang, G. Chen, Z. Wu, L. Yixing, X. Zhou, W. Liu, S. Wang, K. Zhou, R. Shao, L. Nie, Y. Wang, J. Hao, J. Wang, K. Shao, SPA-Bench: A comprehensive benchmark for smartphone agent evaluation, in: The Thirteenth International Conference on Learning Representations (ICLR), 2025.
URL https://openreview.net/forum?id=OZbFRNhpwr

[8] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, M. Fritz, Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection, in: Proceedings of the 2023 ACM SIGSAC Workshop on Artificial Intelligence and Security (AISec), 2023, pp. 79–90. doi:10.1145/3605764.3623985.
URL https://doi.org/10.1145/3605764.3623985

[9] Q. Zhan, Z. Liang, Z. Ying, D. Kang, InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents, in: Findings of the Association for Computational Linguistics: ACL 2024, 2024, pp. 10471–10506. doi:10.18653/v1/2024.findings-acl.624.
URL https://aclanthology.org/2024.findings-acl.624/

[10] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, F. Tramèr, AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents, in: Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track, 2024.
URL https://openreview.net/forum?id=s80yaEXev6

[11] C. Chen, Z. Zhang, B. Guo, S. Ma, I. Khalilov, S. A. Gebreegziabher, Y. Ye, Z. Xiao, Y. Yao, T. Li, T. J.-J. Li, The obvious invisible threat: LLM-powered GUI agents' vulnerability to fine-print injections, arXiv preprint arXiv:2504.11281 (2025). doi:

10.48550/arXiv.2504.11281.
URL https://arxiv.org/abs/2504.11281

[12] Z. Liao, L. Mo, C. Xu, M. Kang, J. Zhang, C. Xiao, Y. Tian, B. Li, H. Sun, EIA: Environmental injection attack on generalist web agents for privacy leakage, arXiv preprint arXiv:2409.11295 (2024). doi:10.48550/arXiv.2409.11295.
URL https://arxiv.org/abs/2409.11295

[13] Y. Chen, X. Hu, K. Yin, J. Li, S. Zhang, Evaluating the robustness of multimodal agents against active environmental injection attacks, arXiv preprint arXiv:2502.13053 (2025). doi:10.48550/arXiv.2502.13053.
URL https://arxiv.org/abs/2502.13053

[14] C. Du, Q. Huang, T. Tang, Z. Wang, A. Nadkarni, Y. Xiao, Exploiting mobile LLM agents as confused deputies via non-app-controlled channels, arXiv preprint arXiv:2510.27140 (2025). doi:10.48550/arXiv.2510.27140.
URL https://arxiv.org/abs/2510.27140

[15] L. Zhao, Z. Zou, S. Li, Z. Liu, Anonymization-enhanced privacy protection for mobile GUI agents: Available but invisible, arXiv preprint arXiv:2602.10139 (2026). doi:10.48550/arXiv.2602.10139.
URL https://arxiv.org/abs/2602.10139

[16] Android Developers, Request SMS verification in an android app, accessed 2026-10-03 (2026).
URL https://developer.android.com/identity/sms-retriever

[17] Android Developers, Autofill framework, accessed 2026-10-03 (2026).
URL https://developer.android.com/identity/autofill

[18] Android Developers, About credential manager, accessed 2026-10-03 (2026).
URL https://developer.android.com/identity/credential-manager

[19] Android Developers, Build autofill services, accessed 2026-10-03 (2026).
URL https://developer.android.com/identity/autofill/autofill-services

[20] Android Developers, Android 15 behavior changes: Security and privacy, accessed 2026-10-03 (2026).
URL https://developer.android.com/about/versions/15/behavior-changes-all

[21] Android Open Source Project, Hardware-backed keystore, accessed 2026-10-03 (2026).
URL https://source.android.com/docs/security/features/keystore

[22] Android Open Source Project, Key and ID attestation, accessed 2026-10-03 (2026).
URL https://source.android.com/docs/security/features/keystore/attestation

[23] Android Open Source Project, Android protected confirmation, accessed 2026-10-03 (2026).
URL https://source.android.com/docs/security/features/protected-confirmation

[24] B. Campbell, J. Bradley, H. Tschofenig, Resource indicators for OAuth 2.0, Tech. Rep. RFC 8707, RFC Editor (2020). doi:10.17487/RFC8707.
URL https://www.rfc-editor.org/info/rfc8707

[25] D. Fett, B. Campbell, J. Bradley, T. Lodderstedt, M. Jones, D. Waite, OAuth 2.0 demonstrating proof of possession (DPoP), Tech. Rep. RFC 9449, RFC Editor (2023). doi:10.17487/RFC9449.
URL https://www.rfc-editor.org/info/rfc9449

[26] W3C Web Authentication Working Group, Web authentication: An API for accessing public key credentials – level 3, W3C Recommendation (2026).
URL https://www.w3.org/TR/webauthn-3/

[27] J. Wang, H. Xu, H. Jia, X. Zhang, M. Yan, W. Shen, J. Zhang, F. Huang, J. Sang, Mobile-agent-v2: Mobile device operation assistant with effective navigation via multi-agent collaboration, in: Advances in Neural Information Processing Systems (NeurIPS), 2024.
URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/052053 7ba799d375b8ff5523295c337a-Abstract-Conference.html

[28] J. Ye, X. Zhang, H. Xu, H. Liu, J. Wang, Z. Zhu, Z. Zheng, F. Gao, J. Cao, Z. Lu, J. Liao, Q. Zheng, F. Huang, J. Zhou, M. Yan, Mobile-agent-v3: Fundamental agents for GUI automation, arXiv preprint arXiv:2508.15144 (2025). doi:10.48550/arXiv .2508.15144.
URL https://arxiv.org/abs/2508.15144

[29] R. Niu, J. Li, S. Wang, Y. Fu, X. Hu, X. Leng, H. Kong, Y. Chang, Q. Wang, ScreenAgent: A vision language model-driven computer control agent, arXiv preprint arXiv:2402.07945 (2024). doi:10.48550/arXiv.2402.07945.
URL https://arxiv.org/abs/2402.07945

[30] T. Xie, D. Zhang, J. Chen, X. Li, S. Zhao, R. Cao, T. J. Hua, Z. Cheng, D. Shin, F. Lei, Y. Liu, Y. Xu, S. Zhou, S. Savarese, C. Xiong, V. Zhong, T. Yu, OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments, in: Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track, 2024.
URL https://openreview.net/forum?id=Mk2A6hJcDp

[31] C. J. Chong, C. Hou, Z. Yao, S. M. S. Talebi, Casper: Prompt sanitization for protecting user privacy in web-based large language models, in: 2025 IEEE 12th International Conference on Cyber Security and Cloud Computing (CSCloud), 2025, pp. 122–131. doi:10.1109/CSCloud66326.2025.00027.
URL https://doi.org/10.1109/CSCloud66326.2025.00027

[32] L. Siyan, V. C. Raghuram, O. Khattab, J. Hirschberg, Z. Yu, PAPILLON: Privacy preservation from internet-based and local language model ensembles, in: Proceedings of NAACL-HLT 2025, 2025, pp. 3371–3390. doi:10.18653/v1/2025.naacl-long.173.
URL https://aclanthology.org/2025.naacl-long.173/

[33] Z. Zeng, J. Wang, J. Yang, Z. Lu, H. Li, H. Zhuang, C. Chen, PrivacyRestore: Privacy-preserving inference in large language models via privacy removal and restoration, in: Proceedings of ACL 2025, 2025, pp. 10821–10855. doi:10.18653/v1/2025.acl-long.532.
URL https://aclanthology.org/2025.acl-long.532/

[34] R. Carpentier, B. Z. H. Zhao, H. J. Asghar, D. Kaafar, Preempting text sanitization utility in resource-constrained privacy-preserving LLM interactions, arXiv preprint arXiv:2411.11521 (2024). doi:10.48550/arXiv.2411.11521.
URL https://arxiv.org/abs/2411.11521

[35] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, F. Tramèr, Defeating prompt injections by design, arXiv preprint arXiv:2503.18813 (2025). doi:10.48550/arXiv.2503.18813.
URL https://arxiv.org/abs/2503.18813

[36] M. Costa, B. Köpf, A. Kolluri, A. Paverd, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, S. Zanella-Béguelin, Securing AI agents with information-flow control, arXiv preprint arXiv:2505.23643 (2025). doi:10.48550/arXiv.2505.23643.
URL https://arxiv.org/abs/2505.23643

[37] T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, D. Song, Progent: Securing AI agents with privilege control, arXiv preprint arXiv:2504.11703 (2025). doi:10.48550/arXiv.2504.11703.
URL https://arxiv.org/abs/2504.11703

[38] Y. Wu, F. Roesner, T. Kohno, N. Zhang, U. Iqbal, IsolateGPT: An execution isolation architecture for LLM-based agentic systems, in: Network and Distributed System Security Symposium, 2025.
URL https://www.ndss-symposium.org/ndss-paper/isolategpt-an-execution-isolation-architecture-for-llm-based-agentic-systems/

[39] E. Bagdasarian, R. Yi, S. Ghalebikesabi, P. Kairouz, M. Gruteser, S. Oh, B. Balle, D. Ramage, AirGapAgent: Protecting privacy-conscious conversational agents, in: Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security, 2024, pp. 3868–3882. doi:10.1145/3658644.3690350.
URL https://doi.org/10.1145/3658644.3690350

[40] J. B. Dennis, E. C. V. Horn, Programming semantics for multiprogrammed computations, in: Communications of the ACM, Vol. 9, 1966, pp. 143–155. doi:10.1145/365230.365252.

[41] J. H. Saltzer, M. D. Schroeder, The protection of information in computer systems, Proceedings of the IEEE 63 (9) (1975) 1278–1308. doi:10.1109/PROC.1975.9939.

[42] R. N. M. Watson, J. Anderson, B. Laurie, K. Kennaway, Capsicum: Practical capabilities for UNIX, in: 19th USENIX Security Symposium, 2010.
URL https://www.usenix.org/conference/usenixsecurity10/capsicum-practical-capabilities-unix

[43] A. P. Felt, E. Chin, S. Hanna, D. Song, D. Wagner, Android permissions demystified, in: Proceedings of the 18th ACM Conference on Computer and Communications Security, 2011, pp. 627–638. doi:10.1145/2046707.2046779.

[44] S. Bugiel, S. Heuser, A.-R. Sadeghi, Flexible and fine-grained mandatory access control on android for diverse security and privacy policies, in: 22nd USENIX Security Symposium, 2013, pp. 131–146.
URL https://www.usenix.org/conference/usenixsecurity13/technical-sessions/paper/bugiel

[45] M. Rossi, D. Facchinetti, E. Bacis, M. Rosa, S. Paraboschi, SEApp: Bringing mandatory access control to android apps, in: 30th USENIX Security Symposium, 2021, pp. 3613–3630.
URL https://www.usenix.org/conference/usenixsecurity21/presentation/rossi

[46] W. Enck, P. Gilbert, B.-G. Chun, L. P. Cox, J. Jung, P. McDaniel, A. N. Sheth, TaintDroid: An information-flow tracking system for realtime privacy monitoring on smartphones, in: 9th USENIX Symposium on Operating Systems Design and Implementation, 2010.
URL https://www.usenix.org/conference/osdi10/taintdroid-information-flow-tracking-system-realtime-privacy-monitoring

[47] L. K. Yan, H. Yin, DroidScope: Seamlessly reconstructing the OS and dalvik semantic views for dynamic android malware analysis, in: 21st USENIX Security Symposium, 2012, pp. 569–584.
URL https://www.usenix.org/conference/usenixsecurity12/technical-sessions/presentation/yan

[48] D. M'Raihi, M. Bellare, F. Hoornaert, D. Naccache, O. Ranen, HOTP: An HMAC-based one-time password algorithm, Tech. Rep. RFC 4226, RFC Editor (2005). doi:10.17487/RFC4226.
URL https://www.rfc-editor.org/info/rfc4226

[49] D. M'Raihi, S. Machani, M. Pei, J. Rydell, TOTP: Time-based one-time password algorithm, Tech. Rep. RFC 6238, RFC Editor (2011). doi:10.17487/RFC6238.
URL https://www.rfc-editor.org/info/rfc6238

[50] National Institute of Standards and Technology, Digital identity guidelines: Authentication and lifecycle management, Tech. Rep. SP 800-63B, NIST (2017). doi:10.6028/NIST.SP.800-63b.
URL https://pages.nist.gov/800-63-3/sp800-63b.html

[51] T. Lodderstedt, J. Bradley, A. Labunets, D. Fett, Best current practice for OAuth 2.0 security, Tech. Rep. RFC 9700, RFC Editor (2025). doi:10.17487/RFC9700.
URL https://www.rfc-editor.org/info/rfc9700

[52] Android Developers, Create your own accessibility service, accessed 2026-10-03 (2026).
URL https://developer.android.com/guide/topics/ui/accessibility/service

[53] Android Developers, One-tap SMS verification with the SMS user consent API, accessed 2026-10-03 (2026).
URL https://developers.google.com/identity/sms-retriever/user-consent/overview

[54] Android Developers, ClipDescription.EXTRA_IS_SENSITIVE api reference, accessed 2026-10-03 (2026).
URL https://developer.android.com/reference/android/content/ClipDescription#EXTRA_IS_SENSITIVE

[55] H. Krawczyk, M. Bellare, R. Canetti, HMAC: Keyed-hashing for message authentication, Tech. Rep. RFC 2104, RFC Editor (1997). doi:10.17487/RFC2104.
URL https://www.rfc-editor.org/info/rfc2104

[56] Android Developers, Manage webview objects, accessed 2026-10-03 (2026).
URL https://developer.android.com/develop/ui/views/layout/webapps/webview

[57] Android Developers, Secure sensitive activities, accessed 2026-10-03 (2026).
URL https://developer.android.com/security/fraud-prevention/activities